

SGPMAIN8.0: Specular Grassmann-Polarity Index Method

*with Transfer Learning, Multi-Objective Prediction, and Peptide
Design*

Carlos Polanco

August 2026

CONTENTS

Contents	i
Glossary of Terms	vii
User Manual	xi
Installation and System Requirements	xiii
Prerequisites	xiii
Required Python Packages	xiii
Installation Steps	xiii
Input File Formats	xv
FASTA File Format	xv
Required Input Files	xv
Configuration Guide	xvii
Setting the Target Group	xvii
Key Configuration Parameters	xviii
ESM2 Configuration	xviii
PIDP Configuration Parameters	xix
Grassmann Multilevel Configuration	xix

Running the Analysis	xxi
Basic Usage	xxi
Example: Analyzing West Nile Virus (WNV)	xxi
Performance Statistics	xxii
Performance Optimization	xxii
Output Files and Their Interpretation	xxiii
Summary of Output Files	xxiii
PIDP Output Files	xxiii
Interpreting PIDP Results	xxiv
Interpreting the Comparison File	xxv
Interpreting Chemical Properties	xxvi
Interpreting Results for Scientific Publications	xxvii
How to Present Results in Papers	xxvii
Adding New Groups	xxxi
Steps to Add a New Protein Group	xxxi
Example: Adding a New Virus Strain	xxxi
Troubleshooting	xxxiii
Common Issues and Solutions	xxxiii
PIDP-Specific Issues	xxxiv
ESM2-Specific Issues	xxxiv
Performance Optimization	xxxiv
Quick Reference Card	xxxv
Key Functions	xxxv
Command Summary	xxxv
File Locations	xxxvi
Important Notes	xxxvi
 I Conceptual Framework and System Architecture	 1
1 Introduction to the SGPMAN8.0 System	3
1.1 Overview and Motivation	3
1.2 System Architecture: Three-Level Design	4
1.3 What's New in Version 8.0	7
 2 The Polarity Index Method (PIM): Mathematical Foundations	 9
2.1 Introduction to the Mathematical Framework	9
2.2 The 16 Polarity Interactions	11
2.3 Biological Weights	11
2.4 Metric Signature	11
2.5 Functions for PIM Computation	12

2.6	Hash Indexing of PIM Vectors	12
3	Grassmann Algebra: Fundamental Operations	13
3.1	Introduction to Grassmann Algebra	13
3.2	Wedge Product and Corrected Similarity	13
3.3	Specular Reflection	14
3.4	Clifford Signature	15
3.5	Interior Product and Subspace Projection	16
3.6	Rotor Operations	17
3.7	Commutator and Anticommutator	17
3.8	Metric Dot Product and Norm	18
3.9	Geometric Product	18
3.10	Hodge Dual	18
3.11	Grassmann Distance and Geodesic	19
II	Mathematical Description of Metrics and Algorithms	21
4	The 18 Mathematical Metrics	23
4.1	Shannon Entropy	23
4.2	Jensen-Shannon Divergence	23
4.3	Gini Coefficient	24
4.4	Structural Complexity Index	24
4.5	Functional Modularity Index	25
4.6	Moran's I	25
4.7	Hellinger Distance	26
4.8	Spearman Correlation	26
4.9	Grassmann Projection Distance	26
4.10	Fubini-Study Metric	27
4.11	Ricci Curvature	27
4.12	Wasserstein Distance	28
4.13	Fractal Dimension	28
4.14	Radon Transform	29
4.15	Karhunen-Loève Decomposition	29
4.16	Polarity Laplacian	30
4.17	Commutator Norm	30
4.18	Anticommutator Similarity	31
5	Transfer Learning with ESM2	33
5.1	Introduction to Protein Language Models	33
5.2	Mathematical Framework	33
5.3	Feature Integration	35
5.4	Mathematical Properties of ESM2 Embeddings	35
6	Multi-Objective Peptide Design	37

6.1	Gaussian Process for Bayesian Optimization	37
6.2	Pareto Front Optimization	38
6.3	Peptide Mutation Strategy	38
6.4	Active Learning Loop	38
7	Drug-Likeness Filters	39
7.1	Lipinski-Modified Rules	39
7.2	Veber Rules	39
7.3	Toxicity Filters	39
7.4	Stability Filters	39
7.5	Synthesis Feasibility	40
7.6	Total Drug-Likeness Score	40
7.7	Recommendation Criteria	40
8	Grassmann Multilevel	41
8.1	grassmann_multilevel_distance	41
8.2	grassmann_multilevel_similarity	41
8.3	grassmann_projection_asymmetry	41
8.4	grassmann_sectional_curvature	42
8.5	grassmann_sectional_curvature_sampled	42
8.6	grassmann_volume	43
8.7	grassmann_cycles	43
8.8	grassmann_karcher_mean	43
8.9	grassmann_svd_similarity	44
9	The Protein Intrinsic Disorder Prediction (PIDP) Module	45
9.1	Motivation and Biological Significance	45
9.2	metapredict: Machine Learning-Based Disorder Prediction	45
9.3	AIUPred: Biophysical Model-Based Disorder Prediction	46
9.4	Integration with the SGPMAN8.0 Workflow	46
9.5	PIDP Output Interpretation	46
10	The ChemicalProfiler Module	49
10.1	Charge Properties (7 properties)	49
10.2	pKa/pI Properties (5 properties)	49
10.3	Electrostatic Properties (4 properties)	49
10.4	Hydrophobicity Properties (4 properties)	50
10.5	Hydrophobic Patches (3 properties)	50
10.6	Glycosylation Sites (2 properties)	50
10.7	PTM Sites (3 properties)	51
10.8	Solubility Properties (3 properties)	51
10.9	Stability Properties (3 properties)	51
10.10	Hotspots (3 properties)	51
10.11	Reactivity Properties (3 properties)	52
10.12	Metal Binding Properties (2 properties)	52

10.13	Membrane Permeability Properties (2 properties)	52
10.14	Lipid Binding Properties (4 properties)	52
10.15	Buffer Stability Properties (4 properties)	53
11	Clases del Sistema - Parte 1: Clases Base y Almacenamiento	55
11.1	DiskCache	55
11.2	SVDCache	55
11.3	ProcessingTracker	55
11.4	OnlineStatistics	56
11.5	ProgressiveSampler	56
11.6	PIMHashIndex	56
11.7	GroupStatistics	57
12	Clases del Sistema - Parte 2: Clases Principales	59
12.1	GrassmannPIM	59
12.2	AdvancedGroupAnalyzer	60
12.3	EnhancedFeatureExtractor	61
12.4	MultiObjectivePredictor	61
12.5	DrugLikenessFilter	62
12.6	PeptideGenerator	62
12.7	PeptideDesignEngine	63
13	ChEMBLMapper	65
13.1	Mapping File Format	65
14	APDLoader	67
14.1	Activity Estimation	67
15	TherapeuticProfiler	69
15.1	Target Identification	69
15.2	Peptide Design	69
15.3	Activity Prediction Model	70
15.4	Physicochemical Properties	70
15.5	Comparison with Known Inhibitors	70
15.6	Recommendations	71
16	The Function main()	73
16.1	Execution Flow	73
16.2	File Loading Order	74
16.3	Results Directory Structure	74
16.4	Command-Line Execution	74
16.5	Error Handling	75

III Case Studies and Applications	77
17 West Nile Virus Case Study	79
17.1 Introduction to West Nile Virus	79
17.2 NILE1: Small Envelope Protein M	79
17.3 NILE2: Envelope Glycoprotein E	80
17.4 PIM Profiles for WNV Proteins	80
17.5 Similarity Analysis: NILE1 vs NILE2	81
17.6 WNV Chemical Profile Summary	81
17.7 Hotspot Analysis	81
17.8 PIDP Analysis for WNV Proteins	82
17.9 Peptide Design for WNV	82
A Reference Database Description	85
A.1 UniProt Groups	85
A.2 Performance Statistics	86
A.3 PIDP Tools	86
A.4 APD Database	86
A.5 Version History	86
B Complete Summary of All 18 Mathematical Metrics	87
C Complete Class and Function Reference	89
C.1 Functions	89
C.2 Classes	91
Bibliography	93

GLOSSARY OF TERMS

Symbol/Term	Meaning	Page
P^+	Positively charged amino acids (H, K, R)	9
P^-	Negatively charged amino acids (D, E)	9
N	Neutral polar amino acids (C, G, N, Q, S, T, Y)	9
NP	Nonpolar amino acids (A, F, I, L, M, P, V, W)	9
v	PIM vector of 16 dimensions	10
$v \wedge w$	Wedge product of v and w	13
$\text{WedgeSim}(v, w)$	Corrected wedge similarity	14
$R_c(v)$	Specular reflection of v	14
$\Sigma(v)$	Clifford signature of v	15
$H(v)$	Shannon entropy of v	23
$\text{JS}(v, w)$	Jensen-Shannon divergence	23
$G(v)$	Gini coefficient	24
$C(v)$	Structural complexity index	24
$M(v)$	Functional modularity index	25
$I(v)$	Moran's I (spatial autocorrelation)	25
$H_d(v, w)$	Hellinger distance	26
$\rho(v, w)$	Spearman correlation	26
λ_k	Karhunen-Loève eigenvalue	29
$d_{Gr}(v, w)$	Grassmann projection distance	26
$d_{FS}(v, w)$	Fubini-Study metric	27
$\kappa(v, w)$	Ricci curvature	27
$W_1(v, w)$	Wasserstein-1 distance	28
$D_f(v)$	Fractal dimension	28
$\mathcal{R}(v)$	Radon transform	29
$L(v)$	Polarity Laplacian	30
$[v, w]$	Commutator norm	30
$\{v, w\}$	Anticommutator similarity	31
$\text{ESM2}(S)$	ESM2 embedding of sequence S	33
$\mathcal{GP}$	Gaussian Process for Bayesian optimization	37
$\mathcal{D}(p)$	Drug-likeness score	39
$\mathcal{P}(p)$	Pareto front of optimal peptides	38
$\mathcal{G}_k(v, w)$	Grassmann multilevel distance (level k)	41
$\mathcal{A}(v, w)$	Grassmann projection asymmetry	41
PIM	Polarity Index Method	9

Symbol/Term	Meaning	Page
GA	Grassmann Algebra	13
SGPMAIN8.0	Specular Grassmann-Polarity Index Method v8.0	3
PIDP	Protein Intrinsic Disorder Prediction	45
GRAVY	Grand Average of Hydropathicity	50
pI	Isoelectric point	50
KL	Karhunen-Loève decomposition	29
IDP	Intrinsically Disordered Protein	45
ChEMBL	ChEMBL protein mapping	65
APD	Antimicrobial Peptide Database	67
LoRA	Low-Rank Adaptation	34
SVD	Singular Value Decomposition	8
UCB	Upper Confidence Bound	37

Note: Page numbers correspond to the first mention of the term in the text.
Terms in **bold** are key concepts of the SGPMAIN system.

Abstract

This document presents the complete user’s manual and conceptual framework of the SGPMAIN8.0 (Specular Grassmann-Polarity Index Method) system, a computational platform for protein analysis and peptide design. The system is built on four fundamental pillars: (1) the Polarity Index Method (PIM) as a representation of proteins as 16-dimensional vectors [1], (2) Grassmann Algebra as the mathematical engine for comparing and transforming polarity profiles [2, 3, 4], (3) Transfer Learning with ESM2 embeddings for enriched sequence representation [24], and (4) Multi-Objective Bayesian Optimization for peptide design [26].

New Features in SGPMAIN8.0:

- **Transfer Learning with ESM2:** Protein language model embeddings (esm2_t6_8M_UR50D) for enhanced feature extraction.
- **Multi-Objective Predictor:** Simultaneous prediction of antiviral activity, cytotoxicity, stability, and selectivity index using ensemble models.
- **Peptide Generator with Bayesian Optimization:** Gaussian Process-based optimization with Upper Confidence Bound (UCB) acquisition.
- **Drug-Likeness Filters:** Lipinski-modified, Veber rules, toxicity, stability, and synthesis feasibility filters.
- **Pareto Front Optimization:** Multi-objective optimization maintaining non-dominated solutions.
- **Active Learning:** Iterative optimization of peptide sequences.
- **LoRA Fine-Tuning:** Efficient fine-tuning of ESM2 using Low-Rank Adaptation.

Existing Features Maintained from v17:

- Grassmann Multilevel (k=1,2,3) with SVD caching
- ChemicalProfiler with 21 unified properties (54 properties total)
- PIDPProfiler (metapredict + AIUPred)
- TherapeuticProfiler (synthetic peptide design)
- ChEMBLMapper and APDLoader
- get_top_individual_proteins()
- All 18 mathematical metrics (see Table 4)

All mathematical formulas are included with clear explanations. Flow diagrams, property tables, and concrete examples using the West Nile virus (WNV) envelope proteins (NILE1 and NILE2) as a case study are included.

Version Information: This manual corresponds to SGPMAIN8.0 (file: SGPMAIN81.0v.py).

User Manual

INSTALLATION AND SYSTEM REQUIREMENTS

Prerequisites

Before installing SGPMAIN8.0, ensure your system meets the following requirements:

Table 2: System requirements for SGPMAIN8.0

Component	Minimum Requirement
Operating System	Linux, macOS, or Windows (with WSL)
Python	Version 3.8 or higher
RAM	8 GB (32 GB recommended for ESM2)
Storage	20 GB free space (for ESM2 models)
Processor	64-bit, 4 or more cores
GPU (optional)	NVIDIA GPU with 4+ GB VRAM for ESM2 acceleration

Required Python Packages

The following Python packages are required:

Table 3: Required Python packages

Package	Version
numpy	$\geq 1.21.0$
pandas	$\geq 1.3.0$
scipy	$\geq 1.7.0$
scikit-learn	$\geq 1.0.0$
matplotlib	$\geq 3.4.0$
seaborn	$\geq 0.11.0$
psutil	$\geq 5.8.0$
torch	$\geq 1.10.0$
transformers	$\geq 4.20.0$
peft	$\geq 0.3.0$

Installation Steps

1. Clone the repository:

```
git clone https://github.com/polancodf2024/nile-sgp-analysis.git
cd nile-sgp-analysis
```

2. Install core dependencies:

```
pip install numpy pandas scipy scikit-learn matplotlib seaborn
psutil
```

3. Install deep learning dependencies:

```
# For PyTorch (CPU version)
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu
```

```
# For PyTorch (GPU version - CUDA 11.8)
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

```
# For transformers and PEFT
pip install transformers peft
```

4. (Optional) Install PIDP dependencies:

```
pip install metapredict aiupred
```

5. Verify the installation:

```
python3 -c 'import torch, transformers, peft;
print("Deep learning OK")'
```

INPUT FILE FORMATS

This chapter describes the input file formats accepted by SGPMAN8.0, including the FASTA format for protein sequences, the ChEMBL mapping file, and the APD peptide database file.

FASTA File Format

SGPMAN8.0 accepts standard FASTA format files with the extension `.fasta` or `.dat0`:

```
>P06935_005 West Nile virus; 11082; 216-290; Small envelope protein M
SLTVQTHGESTLANKKGAWLDSTKATRYLVKTESWILRNPGYALVAAVIGWMLGSNTMQR
VVFALLLLLVAPAYS
>P06935_006 West Nile virus; 11082; 291-787; Envelope protein E
FNCLGMSNRDFLEGVSGATWVDLVLEGDSCVTIMSKDKPTIDVKMMNMEAANLADVRSYC
YLASVSDLSTRAACPTMGEAHNEKRADPAFVCKQGVVDRGWGNGCGLFGKGSIDTCAKFA
...
```

FASTA Format Rules

- Each record starts with a header line beginning with the greater-than symbol
- The header contains the protein identifier and optional description
- Sequence lines follow the header (may span multiple lines)
- Only standard amino acid letters are allowed (A through Z, case-insensitive)

Required Input Files

Table 4: Input files required by SGPMAN8.0

File	Format	Description
*.unico.dat0	FASTA	Protein sequences for each group
chembl_uniprot.txt	TSV	ChEMBL-UniProt mapping (optional)
apd_natural.fasta	FASTA	Antiviral peptide database (optional)

Group File Naming Convention

Each group file should follow the naming pattern:

`{group_name}.unico.dat0`

For example:

- `nile1.unico.dat0` - West Nile virus membrane protein M
- `nile2.unico.dat0` - West Nile virus envelope glycoprotein E
- `lujo.unico.dat0` - Lujo virus envelope glycoprotein

CONFIGURATION GUIDE

This chapter provides a comprehensive guide to configuring SGPMAN8.0, including setting the target group, key configuration parameters, ESM2 settings, PIDP settings, and Grassmann multilevel settings.

Setting the Target Group

The target group is the primary protein or virus family to be analyzed. To change it, modify the `MAIN_GROUP` variable in the configuration section:

Table 5: Examples of target group configurations

Target	Configuration
West Nile virus	<code>MAIN_GROUP = ['nile1', 'nile2']</code>
Lujo virus	<code>MAIN_GROUP = ['lujo']</code>
Ebola virus	<code>MAIN_GROUP = ['zaire', 'sudan', 'reston', 'bundibugyo', 'tai', 'bombali']</code>
Custom proteins	<code>MAIN_GROUP = ['your_protein_group_1', 'your_protein_group_2']</code>

Key Configuration Parameters

Table 6: Key configuration parameters in SGPMAIN8.0

Parameter	Default	Description
MAX_STORED_PROTEINS_PER_GROUP	3000	Maximum proteins stored per group
BATCH_SIZE	30000	Number of sequences per batch
USE_WEIGHTS	True	Enable biological weighting
USE_TRIPLETS	True	Enable tri-vector calculations
USE_QUADRUPLETS	True	Enable quadri-vector calculations
TOP_N_PROTEINS	20	Number of top proteins to return
USE_BOOTSTRAP	True	Enable bootstrap confidence intervals
N_BOOTSTRAP	100	Number of bootstrap iterations

ESM2 Configuration

Table 7: ESM2 configuration parameters

Parameter	Default	Description
ESM2_MODEL_NAME	facebook/esm2_t6_8M_UR50D	ESM2 model variant
ESM2_MAX_LENGTH	1022	Maximum sequence length
ESM2_BATCH_SIZE	8	Batch size for inference
ESM2_USE_GPU	GPU_AVAILABLE	Use GPU if available
ESM2_FINE_TUNE_EPOCHS	10	Fine-tuning epochs
ESM2_LEARNING_RATE	1e-5	Learning rate
ESM2_USE_LORA	True	Use LoRA for efficient fine-tuning
ESM2_LORA_R	8	LoRA rank
ESM2_LORA_ALPHA	16	LoRA alpha
ESM2_LORA_DROPOUT	0.1	LoRA dropout

PIDP Configuration Parameters

Table 8: PIDP configuration parameters

Parameter	Default	Description
USE_PIDP	True	Enable/disable PIDP analysis
PIDP_TARGETS_ONLY	True	Only analyze MAIN_GROUP targets
PIDP_USE_METAPREDICT	True	Use metapredict for disorder prediction
PIDP_USE_AIUPRED	True	Use AIUPred for disorder prediction
PIDP_THRESHOLDS	[0.3, 0.4, 0.5]	Disorder thresholds to report

Grassmann Multilevel Configuration

Table 9: Grassmann multilevel configuration parameters

Parameter	Default	Description
USE_GRASSMANN_MULTILEVEL	True	Enable multilevel Grassmann analysis
GRASSMANN_LEVELS	[1, 2]	Subspace dimensions (k=1,2)
USE_GRASSMANN_ASYMMETRIC	True	Enable asymmetric projection
USE_GRASSMANN_CURVATURE	True	Enable sectional curvature
USE_GRASSMANN_VOLUME	True	Enable simplex volume
USE_GRASSMANN_CYCLES	True	Enable topological cycle detection
USE_GRASSMANN_KARCHER	True	Enable Karcher mean
USE_GRASSMANN_SVD	True	Enable SVD cache

RUNNING THE ANALYSIS

This chapter explains how to execute SGPMIN8.0, provides an example analysis of West Nile virus proteins, and presents performance statistics and optimization strategies.

Basic Usage

The analysis is executed by running the main Python script:

```
python3 SGPMIN81.0v.py
```

The results are saved in a timestamped directory:

```
results_v8_YYYYMMDD_HHMMSS/
```

Example: Analyzing West Nile Virus (WNV)

1. Ensure the following files are in the working directory:

```
nile1.unico.dat0  nile2.unico.dat0
lujo.unico.dat0   lasv_all.unico.dat0
junv_all.unico.dat0  macv_all.unico.dat0
lcmv_all.unico.dat0  CPP.unico.dat0
NON_CPP.unico.dat0  UNFOLDED.unico.dat0
PARTIALLY_FOLDED.unico.dat0
REVIEWED_HUMAN.unico.dat0  UNREVIEWED_HUMAN.unico.dat0
REVIEWED_ALL.unico.dat0  UNREVIEWED_ALL.unico.dat0
VIRUS_REVIEWED.unico.dat0  VIRUS_UNREVIEWED.unico.dat0
SIGNALS.unico.dat0  MEMBRANE.unico.dat0  DISEASE.unico.dat0
```

2. Set `MAIN_GROUP = ['nile1', 'nile2']` in the configuration.
3. Run the analysis:

```
python3 SGPMIN81.0v.py
```

4. Monitor the progress:

```
[OK] Loading ESM2 model: facebook/esm2_t6_8M_UR50D...
[OK] ESM2 loaded successfully on cpu
[OK] Processing NILE1 from nile1.unico.dat0...
[OK] NILE1: 1 valid out of 1 total | Stored: 1 (sample)
[OK] Processing NILE2 from nile2.unico.dat0...
[OK] NILE2: 1 valid out of 1 total | Stored: 1 (sample)
```

Performance Statistics

The system processes 151,066,931 sequences across 26 groups in approximately 13–14 hours.

Table 10: Performance statistics for SGPMAIN8.0

Metric	Value
Total sequences read	151,066,931
Total valid PIMs	151,066,930
Validity rate	100.00%
Total bytes processed	72.17 GB
Processing time	13-14 hours
Average speed	3,141 seq/s
ESM2 model	esm2_t6_8M_UR50D
ESM2 embedding dim	320
Total feature dim	410

Performance Optimization

For large datasets, consider these optimizations:

Table 11: Performance optimization strategies

Strategy	How to implement
Reduce sample size	Set MAX_STORED_PROTEINS_PER_GROUP = 1000
Disable plots	Set GENERATE_PLOTS = False
Disable bootstrap	Set USE_BOOTSTRAP = False
Reduce components	Set USE_TRIPLETS = False
Disable PIDP	Set USE_PIDP = False
Reduce batch size	Set BATCH_SIZE = 50000

OUTPUT FILES AND THEIR INTERPRETATION

This chapter describes the output files generated by SGPMAIN8.0, including the comparison files, PIDP output files, chemical profiles, and how to interpret the results.

Summary of Output Files

File	Description
comparison_{target}_vs_all.csv	Wedge similarity between target and all groups
similarity_matrix_groups.csv	Cross-group similarity matrix (26×26)
top_20_individual_proteins.csv	Top 20 individual proteins most similar to target
chemical_profile_{group}.csv	Complete chemical profile (54 properties)
therapeutic_profile.json	Antiviral peptide design with activity prediction
kl_decomposition.json	Karhunen-Loève decomposition
target_groups_stats_v17.csv	Statistics of target groups
pidp_analysis_{group}.csv	Intrinsic disorder prediction per target
pidp_summary_all_targets.csv	PIDP summary for all targets
pidp_peptide_analysis.csv	PIDP analysis for synthetic peptide
grassmann_multilevel_report.csv	Grassmann multilevel analysis
grassmann_cycles_report.csv	Topological cycle detection
designed_peptides.csv	Designed peptide candidates
best_candidate.json	Best candidate peptide with rationale
design_summary.json	Complete design summary

PIDP Output Files

pidp_analysis_{group}.csv

This file contains the intrinsic disorder prediction results for each target protein using metapredict and AIUPred.

Table 13: Columns in pidp_analysis_{group}.csv

Column	Description
Tool	Prediction tool (metapredict or aiupred)
Metric	Disorder threshold (0.3, 0.4, 0.5) or statistics (mean, max, min, std)
Value	Percentage of disordered residues or statistical value

Example for NILE2:

```

Tool,Metric,Value
metapredict,disorder_0.3,12.33
metapredict,disorder_0.4,9.91
metapredict,disorder_0.5,8.15
metapredict,mean_score,0.1655
aiupred,disorder_0.3,20.26
aiupred,disorder_0.4,12.78
aiupred,disorder_0.5,9.69
aiupred,mean_score,0.1919

```

pidp_summary_all_targets.csv

This file compares PIDP results across all target proteins and the synthetic peptide.

Table 14: Columns in pidp_summary_all_targets.csv

Column	Description
Protein/Peptide	Name of the protein or peptide
Length	Sequence length
Is Peptide	True/False
metapredict_disorder_0.3	Percent disordered at threshold 0.3 (metapredict)
metapredict_disorder_0.4	Percent disordered at threshold 0.4 (metapredict)
metapredict_disorder_0.5	Percent disordered at threshold 0.5 (metapredict)
metapredict_mean	Mean disorder score (metapredict)
aiupred_disorder_0.3	Percent disordered at threshold 0.3 (AIUPred)
aiupred_disorder_0.4	Percent disordered at threshold 0.4 (AIUPred)
aiupred_disorder_0.5	Percent disordered at threshold 0.5 (AIUPred)
aiupred_mean	Mean disorder score (AIUPred)
aiupred_redox	Redox sensitivity (AIUPred)

Interpreting PIDP Results

The interpretation of PIDP results provides guidance on protein structural properties:

- **Low disorder (0 to 10%):** Protein is predominantly ordered. Suitable for crystallization and structure determination.
- **Moderate disorder (10 to 30%):** Protein has flexible regions that may be important for function.
- **High disorder (> 30%):** Protein is largely unstructured. Consider using intrinsically disordered protein (IDP) analysis methods.

For the WNV proteins, the low disorder confirms that they are predominantly ordered proteins, consistent with their roles as structural glycoproteins involved in receptor binding and membrane fusion.

Interpreting the Comparison File

The comparison file contains the following key columns:

Table 15: Columns in the comparison file

Column	Description
Compared Group	Name of the comparison group
Wedge Similarity	Higher values indicate greater similarity (0 to 1 scale)
Wedge Orientation	Positive or negative sign
Probability of Belonging	Statistical probability of group membership
Hydrophobic Angle	Rotation angle in the hydrophobic plane (degrees)
Charge Angle	Rotation angle in the charge plane (degrees)
Specular Reflection	True/False for charge inversion detection
Clifford Distance	Distance in Clifford algebra
Commutator Norm	Norm of the commutator
Anticommutator Sim	Anticommutator similarity
Grassmann Distance	Distance in Grassmann space
Fubini-Study	Fubini-Study metric
Ricci Curvature	Ricci curvature
Wasserstein Distance	Wasserstein-1 distance
Fractal Dim Diff	Fractal dimension difference
Radon Similarity	Radon transform similarity
Laplacian Distance	Laplacian distance
Entropy Diff	Entropy difference
Jensen-Shannon	Jensen-Shannon divergence
Gini Diff	Gini coefficient difference
Spearman	Spearman correlation
Morans I Diff	Moran's I difference

Interpreting Chemical Properties

Table 16: Key chemical properties and their interpretation

Property	Range	Interpretation
Net Charge	-3 to +3	Positive = basic protein, Negative = acidic protein
pI	3 to 10	pH at which net charge is zero
GRAVY	-4.5 to +4.5	Negative = hydrophilic, Positive = hydrophobic
Solubility	1 to 20 mg/mL	Higher = more soluble
Delta G	-15 to -2 kcal/mol	More negative = more stable
Tm	40 to 80 °C	Higher = more thermally stable
PIDP disorder	0 to 100%	Higher = more intrinsically disordered
Glycosylation sites	N-linked, O-linked	Number of predicted sites

INTERPRETING RESULTS FOR SCIENTIFIC PUBLICATIONS

This chapter provides guidance on how to present SGPMAIN8.0 results in scientific publications, including tables, chemical properties, PIDP results, KL decomposition, therapeutic peptide design, and proper citation format.

How to Present Results in Papers

Similarity Tables

When presenting similarity results, include:

1. The target group
2. Wedge similarity values (with 6 decimal places)
3. Orientation signs
4. Hydrophobic and charge angles
5. Number of samples in each group

Example table format from the WNV analysis:

Table 17: Top groups most similar to WNV E protein (NILE2)

Group	Wedge Similarity	Hydro. Angle	Charge Angle
CPP	0.1223	26.45°	29.44°
NON_CPP	0.0738	28.51°	27.51°
NILE1	0.0672	15.30°	30.26°
UNFOLDED	0.0522	12.92°	11.98°

Reporting Chemical Properties

When reporting chemical properties, include:

- Net charge at physiological pH (7.4)
- Isoelectric point (pI)
- GRAVY score
- Predicted stability (Delta G)

- Number of glycosylation sites
- Number of phosphorylation sites
- Lipid binding preferences

Example from the WNV analysis:

The West Nile virus envelope glycoprotein E (NILE2) has a net charge of +0.052 at pH 7.4, a pI of 5.97, and a GRAVY score of 0.253 (slightly hydrophobic). The predicted folding stability is -6.76 kcal/mol with a melting temperature of 41.04 degrees Celsius. The protein has 12 N-linked glycosylation sites, 64 O-linked sites, 32 phosphorylation sites, and 7 acetylation sites. The best lipid binding is cholesterol (score 0.554), followed by ganglioside (0.343) and phosphatidylserine (0.417).

Reporting PIDP Results

When reporting intrinsic disorder prediction results, include:

Intrinsic disorder analysis using metapredict and AIUPred revealed that the WNV envelope glycoprotein E has low disorder propensity with 8.15% of residues predicted as disordered at threshold 0.5 by metapredict and 9.69% by AIUPred. The mean disorder score was 0.1655 for metapredict and 0.1919 for AIUPred.

Reporting KL Decomposition

When reporting KL decomposition results, include:

Karhunen-Loève decomposition revealed that the first component explains 32.49% of the total variance, while the first eight components cumulatively explain 92.45% of the variance, suggesting that the polarity profile follows a structured, low-dimensional organization.

Reporting Therapeutic Peptide Design

When reporting therapeutic peptide design, include:

A competitor peptide (FDLGNNKAAAV) was designed with predicted antiviral activity score of 0.853 (confidence: 0.933), estimated affinity of 0.012 nM, net charge of 0.0, molecular weight of 1209.3 Da, hydrophobicity of 0.018, isoelectric point of 6.0, and solubility of 14.96 mg/mL in PBS buffer.

Citing SGPMAIN8.0

When citing SGPMAIN8.0 in your publications, use the following format:

Polanco, C., Uversky, V.N., et al. (2026). Geometric Algebra-Based Polarity Fingerprinting of West Nile Virus Envelope and Membrane Proteins: A Framework for Therapeutic Peptide Design. *Current Analytical Chemistry*, 2026, (in press).

ADDING NEW GROUPS

This chapter explains how to add new protein groups to the SGPMAN8.0 analysis pipeline.

Steps to Add a New Protein Group

1. Prepare a FASTA file: `your_group.unico.dat0`
2. Add the group name to `files_to_load` in the configuration section.
3. Add the group to `MAIN_GROUP` if it should be a target group.
4. (Optional) Add the group to `GROUP_NAME_MAP` for display.

Example: Adding a New Virus Strain

```
# 1. Add to files_to_load
files_to_load = {
    ...
    'new_virus': 'new_virus.unico.dat0',
    ...
}

# 2. Add to MAIN_GROUP
MAIN_GROUP = ['new_virus', 'nile1', 'nile2']

# 3. Add to GROUP_NAME_MAP
GROUP_NAME_MAP = {
    'new_virus': 'NEW_VIRUS_STRAIN',
}
```


TROUBLESHOOTING

This chapter provides solutions to common issues encountered when running SGPMAIN8.0.

Common Issues and Solutions

Issue	Solution
File not found: *.unico.dat0	Verify the file exists in the working directory
MemoryError	Reduce MAX_STORED_PROTEINS_PER_GROUP
No valid PIM vectors	Check sequence length (greater than 2 amino acids)
Slow execution	Reduce BATCH_SIZE or use fewer groups
LaTeX compilation errors	Use the provided user_manual.tex file
Python import errors	Install missing packages with pip
Permission denied	Use sudo or change file permissions
NameError: GrassmannPIM not defined	Ensure class ordering is correct in SGPMAIN81.0v.py
PIDP tools not available	Install metapredict or aiupred
KL decomposition fails	Check covariance matrix is positive definite
ESM2 model fails to load	Check internet connection or install model locally

PIDP-Specific Issues

Table 19: PIDP-specific troubleshooting

Issue	Solution
metapredict not installed	Install: <code>pip install metapredict</code>
aiupred not installed	Install: <code>pip install aiupred</code>
PIDP analysis skipped	Check <code>USE_PIDP = True</code> in configuration
No sequence for PIDP	Ensure sequences are stored in <code>sample_data</code>
PIDP results missing	Check that target proteins are in <code>MAIN_GROUP</code>
AIUPred redox profile fails	Check aiupred version (≥ 3.0 required)

ESM2-Specific Issues

Table 20: ESM2-specific troubleshooting

Issue	Solution
ESM2 model fails to download	Check internet connection or download manually
GPU out of memory	Set <code>ESM2_USE_GPU = False</code> to use CPU
CUDA not available	Ensure CUDA toolkit is installed
Transformers not installed	<code>pip install transformers[torch]</code>
PEFT not installed	<code>pip install peft</code>

Performance Optimization

- For large datasets, set `MAX_STORED_PROTEINS_PER_GROUP` to 1000
- Disable `GENERATE_PLOTS` for faster execution
- Use `USE_BOOTSTRAP = False` for faster comparisons
- Reduce `N_BOOTSTRAP` from 100 to 50 or 20
- Disable PIDP if not needed: `USE_PIDP = False`
- Reduce `BATCH_SIZE` to 50000 for memory-constrained systems
- For CPU-only systems, use the smaller ESM2 model

QUICK REFERENCE CARD

This chapter provides a quick reference guide to the key functions, commands, file locations, and important notes for SGPMAN8.0.

Key Functions

Table 21: Key functions in SGPMAN8.0

Function	Description
compute_pim_profile(seq)	Convert sequence to PIM vector
wedge_product(v, w)	Calculate wedge similarity
clifford_signature(v)	Calculate Clifford signature
analyze_protein(group)	Full chemical analysis
get_top_individual_proteins(target, n)	Get top N proteins
PIDPProfiler.analyze_sequence(seq, name)	Analyze intrinsic disorder
PIDPProfiler.analyze_target_proteins(dir)	Analyze all target proteins
compute_kl_decomposition(group)	Karhunen-Loève decomposition
design_therapeutic_peptide(target)	Design competitor peptide
PeptideDesignEngine.design_peptide(target, n)	Design optimized peptide

Command Summary

```
# Run full analysis
python3 SGPMAN81.0v.py

# View results
ls results_v8_*/

# Open summary
cat results_v8_*/chemical_profile_nile1.csv

# Open comparison
cat results_v8_*/comparison_nile1_vs_all.csv

# View PIDP results
cat results_v8_*/pidp_summary_all_targets.csv
```

```
# View KL decomposition
cat results_v8_*/kl_decomposition.json

# View therapeutic profile
cat results_v8_*/therapeutic_profile.json

# View designed peptides
cat results_v8_*/designed_peptides.csv
```

File Locations

Table 22: Default file locations

File Type	Location
Input files	Current working directory
Output files	results_v8_YYYYMMDD_HHMMSS/
Log files	results_v8_YYYYMMDD_HHMMSS/log.txt
Configuration	SGPMAIN81.0v.py (edit directly)

Important Notes

1. Always backup your input files before running
2. Results are timestamped to prevent overwriting
3. Large datasets may require several hours to process
4. The program supports Ctrl+C for safe interruption
5. Logs are saved for debugging and reproducibility
6. PIDP requires metapredict and/or aiupred to be installed
7. ESM2 requires PyTorch and transformers to be installed
8. The complete source code is available at:
<https://github.com/polancodf2024/nile-sgp-analysis>

Part I

Conceptual Framework and System Architecture

INTRODUCTION TO THE SGPMAN8.0 SYSTEM

This chapter provides a comprehensive introduction to the SGPMAN8.0 system, including its motivation, the four fundamental pillars on which it is built, its three-level architecture, and the new features introduced in version 8.0.

1.1 Overview and Motivation

The SGPMAN8.0 (Specular Grassmann-Polarity Index Method) system is a computational platform for protein analysis that moves beyond traditional sequence alignment methods to capture the physicochemical essence of proteins. The central insight is that protein function is fundamentally determined by the spatial and temporal organization of polarity transitions along the polypeptide chain, not by the primary sequence of amino acids per se. This insight is crucial because two proteins with completely different sequences can have similar functions if they share similar polarity patterns, and conversely, proteins with similar sequences can have different functions if their polarity patterns diverge.

The system is built on four fundamental pillars that work together to provide a comprehensive analysis framework:

1. **The Polarity Index Method (PIM):** This is the foundational representation layer. It transforms the complex, high-dimensional space of amino acid sequences into a compact 16-dimensional vector space where each dimension corresponds to a specific polarity transition. This transformation is mathematically rigorous yet biologically interpretable, as each transition directly corresponds to a specific physicochemical interaction.
2. **Grassmann Algebra:** This is the computational engine that operates on the PIM vectors. Grassmann algebra provides a natural mathematical framework for comparing, transforming, and analyzing vector subspaces. The key insight is that the wedge product of two PIM vectors gives a measure of their geometric relationship that is both mathematically sound and biologically meaningful.
3. **Transfer Learning with ESM2:** Protein language model embeddings (esm2_t6_8M_UR50D) that capture evolutionary, structural, and functional information from millions of protein sequences. These embeddings provide enriched sequence representation that complements the physicochemical information from PIM.
4. **Multi-Objective Bayesian Optimization:** Gaussian Process-based optimization with Upper Confidence Bound (UCB) acquisition for peptide design. This enables the generation of novel peptides with optimized antiviral activity, cytotoxicity, stability, and selectivity index.

1.2 System Architecture: Three-Level Design

The SGPMAN8.0 system operates through a carefully designed three-level architecture that transforms raw sequence data into actionable biochemical insights. Each level builds upon the previous one, creating a coherent pipeline from sequence to function.

Level 1: Polarity Index Method (PIM) - The Foundation

At the most fundamental level, SGPMAN8.0 implements the Polarity Index Method (PIM). This level takes as input the amino acid sequence of a protein and produces as output a 16-dimensional vector. The transformation is accomplished through the following steps:

First, each amino acid in the sequence is classified into one of four polarity categories based on its physicochemical properties. These categories are: positively charged (P^+ : H, K, R), negatively charged (P^- : D, E), neutral polar (N: C, G, N, Q, S, T, Y), and non-polar hydrophobic (NP: A, F, I, L, M, P, V, W). This classification captures the essential electrostatic and hydrogen-bonding characteristics of each residue.

Second, the sequence is scanned to count the frequency of each possible transition between polarity categories. There are 16 possible transitions (four categories to four categories), and these counts form the components of the PIM vector.

Third, the vector is normalized so that all components sum to one, producing a probability distribution that is independent of sequence length. This normalization ensures that proteins of different lengths can be compared directly.

The result of Level 1 is a compact, length-independent representation of the protein's polarity organization that captures the essential physicochemical information needed for functional analysis. This representation is the foundation upon which all subsequent analyses are built.

Level 2: Grassmann Algebra and Transfer Learning - The Mathematical Engine

At the intermediate level, SGPMAN8.0 applies Grassmann Algebra and Transfer Learning to operate on the PIM vectors produced by Level 1. Grassmann algebra provides a natural mathematical framework for working with vector subspaces and their geometric relationships [2, 3]. ESM2 provides enriched evolutionary and functional context.

The key operations at this level include:

1. The wedge product, which computes the area of the parallelogram spanned by two PIM vectors. This provides a measure of their geometric relationship that captures both similarity and orthogonality.
2. The corrected wedge similarity, which normalizes the wedge product to produce a score between 0 and 1. A score of 1 indicates identical polarity profiles, while a score of 0 indicates orthogonal profiles.
3. Specular reflection, which tests for charge-inverted functional equivalence by inverting the charge components of the PIM vector.

4. The Clifford signature, which combines multiple geometric and information-theoretic measures into a comprehensive fingerprint of the protein’s polarity profile.
5. Karhunen-Loève decomposition, which identifies the most informative components of the polarity profile for dimensionality reduction.
6. ESM2 embedding extraction, which captures evolutionary, structural, and functional information from protein language models.
7. LoRA fine-tuning, which enables efficient adaptation of ESM2 to domain-specific tasks with minimal parameters.

The result of Level 2 is a set of geometric measures and enriched features that describe the relationships between proteins in terms of their polarity organization and evolutionary context.

Level 3: Derived Chemical Properties, PIDP, and Peptide Design - The Biochemical Outputs

At the highest level, SGPMAN8.0 transforms the abstract vector space of PIM vectors into concrete biochemical properties that are directly interpretable by experimental scientists. This level comprises four complementary modules:

Chemical Properties Module: This module derives over 20 physicochemical properties from the PIM vector, including:

- Net charge at physiological pH, which determines electrostatic behavior
- Isoelectric point (pI), the pH at which the protein has zero net charge
- GRAVY (Grand Average of Hydropathicity), a measure of overall hydrophobicity
- Solubility in PBS buffer, which is critical for experimental work
- Conformational stability (ΔG), a measure of folding free energy
- Melting temperature (T_m), indicating thermal stability
- Glycosylation sites (N-linked and O-linked), important for immune evasion
- Phosphorylation sites, indicating regulation by host kinases
- Drug-binding hot spots, potential targets for therapeutic intervention
- Reactivity profile (cysteine, lysine, and tyrosine accessibility)
- Metal binding sites (calcium and zinc)
- Membrane permeability and lipid binding preferences
- Buffer stability recommendations

Protein Intrinsic Disorder Prediction (PIDP) Module: This module provides complementary structural information using two independent methods:

- **metapredict:** A machine learning-based predictor that combines sequence features, evolutionary information, and secondary structure predictions. It produces per-residue disorder scores and percentages of disordered residues at multiple thresholds.
- **AIUPred:** A biophysical model-based predictor that estimates disorder propensity based on amino acid composition and sequence context. It also provides redox sensitivity profiles, indicating how disorder changes under reducing versus oxidizing conditions.

Multi-Objective Predictor Module: This module predicts multiple properties of antiviral peptides:

- Antiviral activity (IC50/EC50)
- Cytotoxicity (CC50)
- Stability (plasma half-life)
- Selectivity index (CC50/IC50)
- Hemolytic activity
- Immunogenicity

Peptide Design Module: This module designs optimized peptides targeting specific proteins:

- Identifies critical polarity transitions between target and host proteins
- Designs peptide sequences that mimic these transitions
- Uses Gaussian Process optimization with UCB acquisition
- Maintains Pareto front of non-dominated solutions
- Evaluates drug-likeness using multiple filters
- Generates synthesis and formulation recommendations

Summary of the Three-Level Architecture

Table 1.1: Summary of the three-level architecture of SGPMAIN8.0

Level	Input	Output
Level 1: PIM	Amino acid sequence	16-dimensional PIM vector
Level 2: Grassmann Algebra + Transfer Learning	PIM vectors	Geometric measures + ESM2 embeddings
Level 3: Derived Properties + PIDP + Peptide Design	PIM vectors + ESM2 embeddings	Chemical properties + PIDP disorder prediction + Therapeutic peptides

This three-level architecture ensures that SGPMAIN8.0 is both mathematically rigorous and practically useful. The abstract mathematics at Levels 1 and 2 provide the foundation for robust, reproducible analysis, while the concrete outputs at Level 3 connect directly to the needs of experimental scientists.

Key Insight: The three levels form a coherent pipeline from sequence to function.

Level 1 transforms sequence into representation.

Level 2 analyzes relationships between representations.

Level 3 translates analysis into actionable biochemical insights.

1.3 What’s New in Version 8.0

SGPMAIN8.0 includes the following new features and improvements over previous versions:

New Features

- Transfer Learning with ESM2:** Integration of protein language model embeddings for enriched sequence representation. The `esm2_t6_8M_UR50D` model provides 320-dimensional embeddings that capture evolutionary, structural, and functional information.
- Multi-Objective Predictor:** Simultaneous prediction of antiviral activity, cytotoxicity, stability, and selectivity index using ensemble models (RandomForest + GradientBoosting).
- Peptide Generator with Bayesian Optimization:** Gaussian Process-based optimization with Upper Confidence Bound (UCB) acquisition for peptide design.

4. **Drug-Likeness Filters:** Lipinski-modified, Veber rules, toxicity, stability, and synthesis feasibility filters for evaluating peptide candidates.
5. **Pareto Front Optimization:** Multi-objective optimization maintaining non-dominated solutions across multiple objectives.
6. **Active Learning:** Iterative optimization of peptide sequences with continuous improvement.
7. **LoRA Fine-Tuning:** Efficient fine-tuning of ESM2 using Low-Rank Adaptation (rank $r=8$, $\alpha=16$, dropout=0.1).

Existing Features Maintained from v17

The following features from version 17 are maintained and enhanced:

1. **Grassmann Multilevel:** Multilevel Grassmann analysis with SVD caching for $k=1,2,3$.
2. **ChemicalProfiler:** 21 unified properties derived from PIM vectors (54 properties total across all categories).
3. **PIDPProfiler:** Integration of metapredict and AIUPred for intrinsic disorder prediction, including redox sensitivity profiles.
4. **TherapeuticProfiler:** Synthetic peptide design with activity prediction using RandomForest models trained on APD data.
5. **ChEMBLMapper:** Mapping of UniProt proteins to ChEMBL targets.
6. **APDLoader:** Loading and processing of antimicrobial peptides from APD database.
7. **get_top_individual_proteins():** Identification of individual proteins most similar to the target.
8. **All 18 Mathematical Metrics:** Shannon entropy, Jensen-Shannon divergence, Gini coefficient, structural complexity, functional modularity, Moran's I, Hellinger distance, Spearman correlation, Grassmann projection, Fubini-Study metric, Ricci curvature, Wasserstein distance, fractal dimension, Radon transform, Karhunen-Loève decomposition, polarity Laplacian, commutator norm, and anticommutator similarity.

THE POLARITY INDEX METHOD (PIM): MATHEMATICAL FOUNDATIONS

This chapter presents the mathematical foundations of the Polarity Index Method (PIM), including the definition of amino acid polarity, the construction of the PIM vector, the 16 polarity interactions, biological weights, the metric signature, and key functions for PIM computation.

2.1 Introduction to the Mathematical Framework

The Polarity Index Method (PIM) is built on a rigorous mathematical foundation that transforms the complex, discrete space of amino acid sequences into a continuous vector space. This transformation is essential because it enables the application of powerful mathematical tools—linear algebra, differential geometry, and information theory—to the analysis of protein sequences. The key insight is that the physicochemical properties of proteins can be captured by the frequencies of transitions between polarity categories, which are independent of the specific amino acid identities.

The mathematical framework has several important properties that make it suitable for protein analysis:

1. Well-posedness: The transformation is uniquely defined for any sequence of sufficient length.
2. Continuity: Small changes in the sequence result in small changes in the PIM vector.
3. Completeness: The transformation captures all information about polarity transitions that is relevant to protein function.
4. Invariance: The transformation is invariant under permutations of the sequence that preserve the transition frequencies.

Definition 2.1.1 (Amino Acid Polarity). *Let $\mathcal{A} = \{A_1, \dots, A_{20}\}$ be the set of the 20 standard amino acids. We define the polarity function $\phi : \mathcal{A} \rightarrow \{P^+, P^-, N, NP\}$ as [1]:*

$$\phi(A) = \begin{cases} P^+ & \text{if } A \in \{H, K, R\} \\ P^- & \text{if } A \in \{D, E\} \\ N & \text{if } A \in \{C, G, N, Q, S, T, Y\} \\ NP & \text{if } A \in \{A, F, I, L, M, P, V, W\} \end{cases} \quad (2.1)$$

Definition 2.1.2 (PIM Vector). For a protein sequence $S = s_1 s_2 \dots s_L$ of length L , we define the PIM vector $v \in \mathbb{R}^{16}$ as:

$$v_i = \frac{1}{L-1} \sum_{k=1}^{L-1} \mathbf{1}\{\phi(s_k) = a, \phi(s_{k+1}) = b\} \quad (2.2)$$

where i is the index corresponding to the transition $a \rightarrow b$ in the order of the 16 possible transitions.

Example 2.1.1. Let's compute the PIM vector for a simple protein sequence: **AKDR**. First, we classify each amino acid:

- *A (Alanine) $\rightarrow$ NP (Nonpolar)*
- *K (Lysine) $\rightarrow$ P^+ (Positively charged)*
- *D (Aspartic acid) $\rightarrow$ P^- (Negatively charged)*
- *R (Arginine) $\rightarrow$ P^+ (Positively charged)*

The polarity string is: **NP, P^+ , P^- , P^+**

Now we count the transitions:

- *NP $\rightarrow$ P^+ : 1 transition*
- *P^+ $\rightarrow$ P^- : 1 transition*
- *$P^- \rightarrow P^+$: 1 transition*

Normalizing by the total number of transitions ($L-1 = 3$), we get the PIM vector $v \in \mathbb{R}^{16}$. The indices for the transitions are:

- *NP $\rightarrow$ P^+ : index 12 (0-based counting: NP=3, P^+ =0, so $3 \times 4 + 0 = 12$)*
- *$P^+ \rightarrow P^-$: index 1 ($0 \times 4 + 1 = 1$)*
- *$P^- \rightarrow P^+$: index 4 ($1 \times 4 + 0 = 4$)*

Thus:

$$v_{12} = 1/3, \quad v_1 = 1/3, \quad v_4 = 1/3 \quad (2.3)$$

and all other components are 0.

Solución: The PIM vector is:

$$v = [0 \quad 1/3 \quad 0 \quad 0 \quad 1/3 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1/3 \quad 0 \quad 0 \quad 0]^T \quad (2.4)$$

□

2.2 The 16 Polarity Interactions

The 16 possible polarity transitions are:

Table 2.1: The 16 polarity transitions with indices

Index	Transition	Index	Transition
0	$P^+ \rightarrow P^+$	8	$N \rightarrow P^+$
1	$P^+ \rightarrow P^-$	9	$N \rightarrow P^-$
2	$P^+ \rightarrow N$	10	$N \rightarrow N$
3	$P^+ \rightarrow NP$	11	$N \rightarrow NP$
4	$P^- \rightarrow P^+$	12	$NP \rightarrow P^+$
5	$P^- \rightarrow P^-$	13	$NP \rightarrow P^-$
6	$P^- \rightarrow N$	14	$NP \rightarrow N$
7	$P^- \rightarrow NP$	15	$NP \rightarrow NP$

2.3 Biological Weights

The system uses biological weights to emphasize physiologically relevant interactions:

Table 2.2: Biological weights for polarity interactions

Interaction	Weight
P^+, P^-	2.0
P^-, P^+	2.0
N, N	1.5
N, P^+	1.3
P^+, N	1.3
N, P^-	1.3
P^-, N	1.3
NP, NP	1.0
NP, N	0.9
N, NP	0.9
NP, P^+	0.7
P^+, NP	0.7
NP, P^-	0.7
P^-, NP	0.7
P^+, P^+	0.4
P^-, P^-	0.4

2.4 Metric Signature

The metric signature defines the geometry of the PIM space:

$$\text{METRIC_SIGNATURE} = \begin{cases} [-1, +1, +1, 0, +1, -1, +1, 0, \\ +1, +1, +1, 0, 0, 0, 0, +1] \\ \text{if USE_BIOLOGICAL_METRIC} \\ [1, 1, 1, 1, 1, 1, 1, 1, \\ 1, 1, 1, 1, 1, 1, 1, 1] \\ \text{if not} \end{cases} \quad (2.5)$$

2.5 Functions for PIM Computation

This section describes the key functions used for computing PIM profiles and related features.

`compute_pim_profile`

This function converts a protein sequence into a PIM vector:

$$\text{pim_profile}(S) = \frac{1}{L-1} \sum_{k=1}^{L-1} \mathbf{1}\{\phi(s_k) \rightarrow \phi(s_{k+1})\} \quad (2.6)$$

The function also applies biological weights if `USE_WEIGHTS` is set to `True`. The weighted vector is normalized so that all components sum to one.

`compute_trimer_profile`

This function computes the trimer profile for a sequence:

$$\text{trimer_profile}(S) = \frac{1}{L-2} \sum_{k=1}^{L-2} \mathbf{1}\{\phi(s_k) \rightarrow \phi(s_{k+1}) \rightarrow \phi(s_{k+2})\} \quad (2.7)$$

2.6 Hash Indexing of PIM Vectors

The system uses LSH (Locality-Sensitive Hashing) for efficient similarity search. The `pim_to_hash` function discretizes the PIM vector with a given tolerance and generates a SHA-256 hash of the discretized vector. This enables fast retrieval of similar proteins.

```
pim_to_hash(v, tolerance) -> hash_string
```


GRASSMANN ALGEBRA: FUNDAMENTAL OPERATIONS

This chapter presents the fundamental operations of Grassmann algebra as implemented in SGPMAIN8.0, including the wedge product, specular reflection, Clifford signature, interior product, rotor operations, commutator and anticommutator, metric dot product, geometric product, Hodge dual, and Grassmann distance.

3.1 Introduction to Grassmann Algebra

Grassmann algebra, also known as exterior algebra, provides a natural mathematical framework for working with vector subspaces and their geometric relationships [2, 3]. The fundamental operation in Grassmann algebra is the wedge product, which generalizes the cross product to arbitrary dimensions. This is essential for protein analysis because it allows us to measure the geometric relationship between two PIM vectors in a way that captures both their similarity and their orthogonality.

The key insight of Grassmann algebra is that while vectors represent points or directions, the wedge product of two vectors represents a plane (or more generally, a subspace). The magnitude of the wedge product gives the area of the parallelogram spanned by the vectors, and its direction gives the orientation of that parallelogram. This geometric interpretation is particularly powerful for protein analysis because it allows us to think of PIM vectors as directions in a 16-dimensional space, and the wedge product as a measure of how aligned those directions are.

3.2 Wedge Product and Corrected Similarity

Mathematical Definition and Interpretation

The wedge product of two PIM vectors captures the geometric relationship between them. The full wedge product in 16 dimensions would have 120 components, but we use a truncated version that focuses on the six biologically most relevant pairs of transitions. This truncation is justified because the six selected pairs capture the essential physicochemical information needed for functional comparison.

Definition 3.2.1 (Truncated Wedge Product). *Let $v, w \in \mathbb{R}^{16}$ be two PIM vectors. We define the truncated wedge product over the set of biologically relevant pairs $\mathcal{K}$:*

$$\mathcal{K} = \{(0, 5), (1, 4), (2, 6), (3, 7), (10, 11), (14, 15)\} \quad (3.1)$$

The truncated wedge product is:

$$\omega_{\mathcal{K}}(v, w)_k = v_{i_k} w_{j_k} - v_{j_k} w_{i_k} \quad \text{for } (i_k, j_k) \in \mathcal{K} \quad (3.2)$$

Definition 3.2.2 (Corrected Wedge Similarity). *The corrected wedge similarity between two PIM vectors $v, w \in \mathbb{R}^{16}$ is defined as:*

$$\text{WedgeSim}(v, w) = 1 - \frac{\|\omega_{\mathcal{K}}(v, w)\|_2}{\|v\|_2 \|w\|_2} \quad (3.3)$$

This similarity satisfies $0 \leq \text{WedgeSim}(v, w) \leq 1$, where $\text{WedgeSim}(v, w) = 1$ indicates identical vectors and $\text{WedgeSim}(v, w) = 0$ indicates orthogonal vectors.

Remark 3.2.1. *The corrected wedge similarity is a more stringent measure than the uncorrected wedge similarity used in previous versions. A value of 0.12 indicates near-orthogonality of the polarity profiles, representing a substantial functional difference between the compared proteins. This improved discriminative power is the key advance of version 17.*

General Wedge Product

For higher-grade wedge products, the following definitions apply:

For grade 2 (bivector):

$$(v \wedge w)_{ij} = v_i w_j - v_j w_i \quad (3.4)$$

For grade 3 (trivector):

$$(v \wedge w)_{ijk} = \det \begin{pmatrix} v_i & w_i & 1 \\ v_j & w_j & 1 \\ v_k & w_k & 1 \end{pmatrix} \quad (3.5)$$

For grade 4 (quadrivector):

$$(v \wedge w)_{ijkl} = \det \begin{pmatrix} v_i & w_i & 1 & 1 \\ v_j & w_j & 1 & 1 \\ v_k & w_k & 1 & 1 \\ v_l & w_l & 1 & 1 \end{pmatrix} \quad (3.6)$$

3.3 Specular Reflection

The Concept of Charge Inversion

Specular reflection is a novel concept introduced in SGPMAIN8.0 that captures the idea of charge-inverted functional equivalence. The insight is that two proteins might have similar

functions even if their charge patterns are inverted—a phenomenon that occurs in some cases of molecular mimicry and immune evasion. The mathematical operation of specular reflection inverts the charge components of the PIM vector, allowing us to test whether two proteins are related by charge inversion.

The mathematical properties of specular reflection ensure that it is a well-defined geometric transformation:

1. The reflection is an involution (applying it twice returns the original vector).
2. The reflection preserves the Euclidean norm (it is an isometry).
3. The reflection is self-adjoint (it preserves the inner product structure).

Definition 3.3.1 (Reflection Normal Vector). *The normal vector $n \in \mathbb{R}^{16}$ for specular reflection is defined as [10]:*

$$n_i = \begin{cases} +1 & \text{if } i \in \{0, 1, 4, 5\} \text{ (charge components)} \\ 0 & \text{otherwise} \end{cases} \quad (3.7)$$

normalized as $n = n/\|n\|_2$.

Definition 3.3.2 (Specular Reflection). *The specular reflection of a PIM vector $v \in \mathbb{R}^{16}$ is defined as:*

$$R_c(v) = v - 2(v \cdot n)n \quad (3.8)$$

This transformation inverts the sign of the charge components, simulating a charge inversion (P^+ to P^- and P^- to P^+).

3.4 Clifford Signature

Comprehensive Characterization

The Clifford signature provides a comprehensive characterization of a protein’s polarity profile by combining multiple geometric, information-theoretic, and statistical measures into a single vector [2, 3]. Each component of the signature captures a different aspect of the polarity profile, and together they form a fingerprint that can be used for classification, comparison, and functional prediction.

The components of the Clifford signature are chosen to be:

1. Informative: Each component provides unique information about the polarity profile.
2. Interpretable: Each component has a clear biological meaning.
3. Stable: Each component is robust to small changes in the sequence.

4. Discriminative: Each component helps to distinguish between different functional classes of proteins.

Definition 3.4.1 (Clifford Signature). *The Clifford signature of a PIM vector $v \in \mathbb{R}^{16}$ is the 12-component vector:*

$$\Sigma(v) = (\|v\|, \text{AutoRef}(v), \text{HydroProj}(v), \text{ChargeProj}(v), \text{AutoRot}(v), H(v), D_f(v), G(v), C(v), M(v), I(v), \Lambda(v)) \quad (3.9)$$

where:

$$\text{AutoRef}(v) = \text{WedgeSim}(v, R_c(v)) \quad (3.10)$$

$$\text{HydroProj}(v) = |v_{10} + v_{15}| \quad (3.11)$$

$$\text{ChargeProj}(v) = |v_0 + v_5| \quad (3.12)$$

$$\text{AutoRot}(v) = \text{WedgeSim}(v, \text{roll}(v, 4)) \quad (3.13)$$

3.5 Interior Product and Subspace Projection

The interior product measures the projection of a vector onto a subspace:

$$\text{proj}(v, \text{subspace}) = v \text{ restricted to the indices of the subspace} \quad (3.14)$$

$$\|\text{proj}(v, \text{subspace})\| \quad (3.15)$$

The predefined subspaces are:

Table 3.1: Predefined subspaces for interior product

Subspace	Indices
hydrophobic	[10, 15]
charge_repulsion	[0, 5]
charge_attraction	[1, 4]
charge_polar	[2, 3, 6, 7]
polar	[8, 9, 10, 11]
nonpolar	[12, 13, 14, 15]
full	All indices

3.6 Rotor Operations

Rotor Angle

The rotor angle measures the angle between two vectors in a specific plane:

$$\theta = \arccos \left(\frac{v_i w_i + v_j w_j}{\sqrt{v_i^2 + v_j^2} \sqrt{w_i^2 + w_j^2}} \right) \quad (3.16)$$

The predefined rotor planes are:

Table 3.2: Predefined rotor planes

Name	Indices	Description
hydrophobic	(10, 15)	N→N vs NP→NP
charge	(0, 5)	P ⁺ →P ⁺ vs NP→NP
opposite_charge	(1, 4)	P ⁺ →P ⁻ vs P ⁻ →P ⁺
polarity	(10, 11)	N→N vs N→NP
charge_transition	(2, 8)	P ⁺ →N vs N→P ⁺
opposite_transition	(6, 9)	P ⁻ →N vs N→P ⁻

General Rotor

The general rotor interpolates between two vectors:

$$v(t) = \cos(t\theta)v + \sin(t\theta)\text{plane} \quad (3.17)$$

3.7 Commutator and Anticommutator

Definition 3.7.1 (Commutator). *The commutator of two PIM vectors is:*

$$[v, w] = v \wedge w - w \wedge v = 2(v \wedge w) \quad (3.18)$$

The commutator norm is:

$$\|[v, w]\| = \frac{\|v \wedge w\|}{\|v\| \|w\|} \quad (3.19)$$

Definition 3.7.2 (Anticommutator). *The anticommutator of two PIM vectors is:*

$$\{v, w\} = v \cdot w + w \cdot v = 2(v \cdot w) \quad (3.20)$$

The anticommutator similarity is:

$$\text{anticommutator_sim}(v, w) = \frac{|v \cdot w|}{\|v\| \|w\|} \quad (3.21)$$

3.8 Metric Dot Product and Norm

Dot Product with Metric

$$\langle v, w \rangle_\eta = \sum_i \eta_i v_i w_i \quad (3.22)$$

Norm with Metric

$$\|v\|_\eta = \sqrt{|\langle v, v \rangle_\eta|} \quad (3.23)$$

Similarity with Metric

$$\text{sim}_\eta(v, w) = \frac{|\langle v, w \rangle_\eta|}{\|v\|_\eta \|w\|_\eta} \quad (3.24)$$

3.9 Geometric Product

Full Geometric Product

The full geometric product decomposes into grades:

$$vw = \langle v, w \rangle + v \wedge w + v \wedge w \wedge (\text{trivector}) + (\text{quadrivector}) \quad (3.25)$$

Functional-Structural Decomposition

$$\text{functional} = \frac{|\langle v, w \rangle|}{\|v\| \|w\|}, \quad \text{structural} = \frac{\|v \wedge w\|}{\|v\| \|w\|} \quad (3.26)$$

3.10 Hodge Dual

Hodge Dual Operation

$$\star v = \text{orthogonal complement of } v \quad (3.27)$$

Hodge Complementarity

$$\text{hodge_comp}(v, w) = \text{WedgeSim}(w, \star v) \quad (3.28)$$

3.11 Grassmann Distance and Geodesic

Grassmann Distance

$$d_G(v, w) = \frac{1}{\sqrt{2}} \|P_v - P_w\|_F, \quad P_v = \frac{vv^T}{\|v\|^2} \quad (3.29)$$

Grassmann Geodesic

$$v(t) = \frac{\sin((1-t)\theta)}{\sin(\theta)} v + \frac{\sin(t\theta)}{\sin(\theta)} w \quad (3.30)$$

Part II

Mathematical Description of Metrics and Algorithms

THE 18 MATHEMATICAL METRICS

This chapter presents the complete mathematical description of all 18 metrics implemented in SGPMAIN8.0. These metrics are derived from the PIM vectors and provide a comprehensive characterization of protein polarity profiles. Each metric captures a different aspect of the polarity distribution, from information-theoretic measures to geometric and topological properties.

4.1 Shannon Entropy

The Shannon entropy measures the uncertainty or randomness in the polarity distribution [11]. A higher entropy indicates a more uniform distribution of polarity transitions, while a lower entropy indicates that a few transitions dominate. This metric is fundamental for characterizing the diversity of polarity interactions in a protein.

Definition 4.1.1 (Shannon Entropy). *For a PIM vector $v \in \mathbb{R}^{16}$, the Shannon entropy is:*

$$H(v) = - \sum_i \frac{|v_i|}{\|v\|_1} \log_2 \left(\frac{|v_i|}{\|v\|_1} + \epsilon \right) \quad (4.1)$$

where $\|v\|_1 = \sum_i |v_i|$ is the L1 norm, and $\epsilon = 10^{-10}$ is a small constant to avoid division by zero.

The entropy ranges from 0 to $\log_2(16) = 4$ bits.

4.2 Jensen-Shannon Divergence

The Jensen-Shannon divergence measures the similarity between two probability distributions [12, 11]. It is a symmetric and smoothed version of the Kullback-Leibler divergence. Unlike the KL divergence, the JS divergence is symmetric and always finite, making it particularly suitable for comparing protein polarity profiles.

Definition 4.2.1 (Jensen-Shannon Divergence). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Jensen-Shannon divergence is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon}, \quad q = \frac{|w|}{\|w\|_1 + \epsilon} \quad (4.2)$$

$$m = \frac{p + q}{2} \quad (4.3)$$

$$JS(v, w) = \frac{1}{2} \sum_i p_i \log_2 \left(\frac{p_i}{m_i} \right) + \frac{1}{2} \sum_i q_i \log_2 \left(\frac{q_i}{m_i} \right) \quad (4.4)$$

The JS divergence ranges from 0 (identical distributions) to 1 (completely different distributions).

4.3 Gini Coefficient

The Gini coefficient measures the inequality in the polarity distribution [13]. A lower Gini coefficient indicates a more uniform distribution, while a higher coefficient indicates that the distribution is concentrated in a few transitions. This metric is particularly useful for identifying proteins with highly specialized polarity patterns.

Definition 4.3.1 (Gini Coefficient). *For a PIM vector $v \in \mathbb{R}^{16}$, the Gini coefficient is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon} \quad (4.5)$$

$$G(v) = 1 - \frac{2}{n} \sum_{i=1}^n \frac{\sum_{j \leq i} p(j)}{\sum_j p(j)} \quad (4.6)$$

where $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(n)}$ are the sorted probabilities, and $n = 16$.

The Gini coefficient ranges from 0 (perfect equality) to 1 (perfect inequality).

4.4 Structural Complexity Index

The structural complexity index combines multiple measures to capture the overall complexity of the polarity profile. This composite metric integrates entropy, fractal dimension, and richness to provide a single value that reflects the structural complexity of the protein's polarity organization.

Definition 4.4.1 (Structural Complexity Index). *For a PIM vector $v \in \mathbb{R}^{16}$, the structural complexity index is:*

$$C(v) = 0.4H_{norm}(v) + 0.35D_f(v) + 0.25R(v) \quad (4.7)$$

where:

- $H_{norm}(v) = H(v)/\log_2(16)$ is the normalized Shannon entropy
- $D_f(v)$ is the fractal dimension (see Section 7.13)
- $R(v) = nnz(v)/16$ is the richness measure (number of non-zero components)

The complexity index ranges from 0 to 1.

4.5 Functional Modularity Index

The functional modularity index measures the degree of clustering in the polarity transitions. This metric identifies whether significant polarity transitions are organized into distinct clusters, which may correspond to functional domains or regions of the protein.

Definition 4.5.1 (Functional Modularity Index). *For a PIM vector $v \in \mathbb{R}^{16}$ and a threshold $\theta = 0.05$, the functional modularity index is:*

1. *Identify significant transitions: $S = \{i \mid |v_i| > \theta\}$*
2. *Cluster significant transitions by proximity: clusters are formed when indices differ by at most 2*
3. *Compute cluster energies: $E_k = \sum_{i \in \text{cluster}_k} |v_i|$*
4. *Normalize: $\hat{E}_k = E_k / \sum_j E_j$*
5. *Compute entropy: $H_{\text{cluster}} = - \sum_k \hat{E}_k \log_2(\hat{E}_k)$*

$$M(v) = 1 - \frac{H_{\text{cluster}}}{\log_2(K)} \quad (4.8)$$

where K is the number of clusters. The modularity index ranges from 0 (no modularity) to 1 (perfect modularity).

4.6 Moran's I

Moran's I measures the spatial autocorrelation in the polarity profile [14]. Positive values indicate clustering of similar values, negative values indicate dispersion, and zero indicates random distribution. This metric captures the spatial organization of polarity transitions along the sequence.

Definition 4.6.1 (Moran's I). *For a PIM vector $v \in \mathbb{R}^{16}$, Moran's I is:*

$$z_i = v_i - \bar{v} \quad (4.9)$$

$$w_{ij} = \exp\left(-\frac{|i-j|}{2}\right) \quad (4.10)$$

$$I(v) = \frac{n}{\sum_{i,j} w_{ij}} \frac{\sum_{i,j} w_{ij} z_i z_j}{\sum_i z_i^2} \quad (4.11)$$

where $n = 16$ is the number of components, and $\bar{v}$ is the mean of v .

4.7 Hellinger Distance

The Hellinger distance measures the similarity between two probability distributions [15]. It is a metric that satisfies the triangle inequality and is bounded between 0 and 1, making it particularly suitable for comparing protein polarity profiles.

Definition 4.7.1 (Hellinger Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Hellinger distance is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon}, \quad q = \frac{|w|}{\|w\|_1 + \epsilon} \quad (4.12)$$

$$H_d(v, w) = \frac{1}{\sqrt{2}} \|\sqrt{p} - \sqrt{q}\|_2 \quad (4.13)$$

The Hellinger distance ranges from 0 (identical distributions) to 1 (completely different distributions).

4.8 Spearman Correlation

The Spearman correlation measures the rank correlation between two PIM vectors [16]. Unlike the Pearson correlation, the Spearman correlation is non-parametric and captures monotonic relationships, making it robust to outliers and non-linear relationships.

Definition 4.8.1 (Spearman Correlation). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Spearman correlation is:*

$$r_i = \text{rank}(v_i), \quad s_i = \text{rank}(w_i) \quad (4.14)$$

$$\rho(v, w) = \frac{\sum_i (r_i - \bar{r})(s_i - \bar{s})}{\sqrt{\sum_i (r_i - \bar{r})^2 \sum_i (s_i - \bar{s})^2}} \quad (4.15)$$

The Spearman correlation ranges from -1 (perfect negative correlation) to 1 (perfect positive correlation).

4.9 Grassmann Projection Distance

The Grassmann projection distance measures the distance between two subspaces in the Grassmann manifold [5]. This metric captures the geometric relationship between the subspaces spanned by two PIM vectors, providing a measure of their structural similarity.

Definition 4.9.1 (Grassmann Projection Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Grassmann projection distance is:*

$$P_v = \frac{vv^T}{\|v\|^2}, \quad P_w = \frac{ww^T}{\|w\|^2} \quad (4.16)$$

$$d_{Gr}(v, w) = \frac{1}{\sqrt{2}} \|P_v - P_w\|_F \quad (4.17)$$

where $\|\cdot\|_F$ is the Frobenius norm. The distance ranges from 0 (identical subspaces) to 1 (orthogonal subspaces).

4.10 Fubini-Study Metric

The Fubini-Study metric measures the distance in projective space [6, 7]. This metric captures the angle between two vectors in the projective space, providing a measure of their directional similarity.

Definition 4.10.1 (Fubini-Study Metric). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Fubini-Study metric is:*

$$d_{FS}(v, w) = \arccos \left(\frac{|v \cdot w|}{\|v\|_2 \|w\|_2} \right) \quad (4.18)$$

The Fubini-Study metric ranges from 0 (identical directions) to $\pi/2$ (orthogonal directions).

4.11 Ricci Curvature

The Ricci curvature measures the geometric curvature of the space between two vectors. This metric captures the local geometry of the PIM space and can indicate regions of high or low curvature that may correspond to functional significance.

Definition 4.11.1 (Ricci Curvature). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Ricci curvature is:*

$$\theta = \arccos \left(\frac{|v \cdot w|}{\|v\|_2 \|w\|_2} \right) \quad (4.19)$$

$$\kappa(v, w) = \frac{1}{\tan^2(\theta) + \epsilon} \quad (4.20)$$

where $\epsilon = 10^{-10}$ is a small constant to avoid division by zero. Higher curvature indicates more curved geometry.

4.12 Wasserstein Distance

The Wasserstein-1 distance measures the optimal transport cost between two distributions [17]. This metric captures the minimal cost of transforming one polarity distribution into another, providing a meaningful measure of distributional similarity.

Definition 4.12.1 (Wasserstein-1 Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the Wasserstein-1 distance is:*

$$p = \frac{|v|}{\|v\|_1 + \epsilon}, \quad q = \frac{|w|}{\|w\|_1 + \epsilon} \quad (4.21)$$

$$CDF_v(i) = \sum_{j=1}^i p_j, \quad CDF_w(i) = \sum_{j=1}^i q_j \quad (4.22)$$

$$W_1(v, w) = \sum_{i=1}^n |CDF_v(i) - CDF_w(i)| \quad (4.23)$$

where $n = 16$. The Wasserstein distance ranges from 0 (identical distributions) to 16 (completely different distributions).

4.13 Fractal Dimension

The fractal dimension measures the self-similarity of the polarity profile [9]. This metric captures the scale-invariant properties of the polarity distribution, providing insight into its structural complexity.

Definition 4.13.1 (Fractal Dimension). *For a PIM vector $v \in \mathbb{R}^{16}$, the fractal dimension is:*

1. Compute the absolute values: $a_i = |v_i|$
2. Compute the cumulative sum: $c_i = \sum_{j=1}^i a_j$
3. Normalize: $\hat{c}_i = (c_i - \min c) / (\max c - \min c)$
4. For each scale $s = 1, \dots, S$, compute box count: $N_s = \#\{\text{unique boxes}\}$

$$D_f(v) = -\frac{\log(N_s)}{\log(s)} \quad (4.24)$$

The fractal dimension ranges from 0 to 1.

4.14 Radon Transform

The Radon transform projects the polarity profile onto different angles [8]. This transformation captures the directional properties of the polarity distribution, providing a representation that is invariant to certain types of transformations.

Definition 4.14.1 (Radon Transform). *For a PIM vector $v \in \mathbb{R}^{16}$, the Radon transform is:*

$$\mathcal{R}(v)(\theta_k) = \sum_{i=1}^n v_i^2 \cos^2(i\theta_k) \quad (4.25)$$

where $\theta_k = k\pi/n$ for $k = 0, \dots, n-1$, and $n = 16$.

The Radon similarity between two vectors is:

$$\text{RadonSim}(v, w) = \frac{\mathcal{R}(v) \cdot \mathcal{R}(w)}{\|\mathcal{R}(v)\|_2 \|\mathcal{R}(w)\|_2} \quad (4.26)$$

4.15 Karhunen-Loève Decomposition

The Karhunen-Loève decomposition (also known as Principal Component Analysis) identifies the most informative components of the polarity profile [18, 19]. This decomposition provides a low-dimensional representation that preserves the essential information about the polarity distribution.

Definition 4.15.1 (Karhunen-Loève Decomposition). *Let $\{v_1, \dots, v_n\}$ be a set of PIM vectors. The covariance matrix is:*

$$C = \frac{1}{n-1} \sum_{i=1}^n (v_i - \bar{v})(v_i - \bar{v})^T \quad (4.27)$$

where $\bar{v}$ is the mean vector.

The Karhunen-Loève decomposition solves the eigenvalue problem:

$$C\phi_k = \lambda_k\phi_k, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{16} \geq 0 \quad (4.28)$$

The explained variance for component k is:

$$\text{EV}_k = \frac{\lambda_k}{\sum_{j=1}^{16} \lambda_j} \quad (4.29)$$

The cumulative explained variance for the first K components is:

$$\text{EV}_{cum}(K) = \sum_{k=1}^K \text{EV}_k \quad (4.30)$$

Example 4.15.1. *For the WNV proteins, the Karhunen-Loève decomposition revealed that the first component explains 32.49% of the total variance, while the first eight components cumulatively explain 92.45% of the variance. The rapid decay of eigenvalues confirms that the polarity profile follows a structured, low-dimensional organization.*

4.16 Polarity Laplacian

The polarity Laplacian measures the connectivity structure of the polarity profile. This metric captures the graph-theoretic properties of the polarity transitions, providing insight into the network structure of polarity interactions.

Definition 4.16.1 (Polarity Laplacian). *For a PIM vector $v \in \mathbb{R}^{16}$, the polarity Laplacian is:*

$$L_{ij} = \begin{cases} \sum_{k \neq i} v_i v_k & \text{if } i = j \\ -v_i v_j & \text{if } i \neq j \end{cases} \quad (4.31)$$

The Laplacian properties include:

- *Spectral gap: $\lambda_2 - \lambda_1$ (the difference between the second and first eigenvalues)*
- *Connectivity: $\frac{\#\{(i,j) | L_{ij} \neq 0\}}{n(n-1)/2}$*
- *Fiedler vector: the eigenvector corresponding to λ_2*

4.17 Commutator Norm

The commutator norm measures the non-commutativity of two PIM vectors. This metric captures the degree to which the wedge product fails to commute, providing a measure of the geometric incompatibility between two polarity profiles.

Definition 4.17.1 (Commutator Norm). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the commutator norm is:*

$$[v, w] = v \wedge w - w \wedge v = 2(v \wedge w) \quad (4.32)$$

$$\|[v, w]\| = \frac{\|v \wedge w\|_2}{\|v\|_2 \|w\|_2} \quad (4.33)$$

The commutator norm ranges from 0 (commuting vectors) to 1 (completely non-commuting vectors).

4.18 Anticommutator Similarity

The anticommutator similarity measures the commutativity of two PIM vectors. This metric captures the degree to which the dot product captures the relationship between two polarity profiles.

Definition 4.18.1 (Anticommutator Similarity). *For two PIM vectors $v, w \in \mathbb{R}^{16}$, the anticommutator similarity is:*

$$\{v, w\} = v \cdot w + w \cdot v = 2(v \cdot w) \quad (4.34)$$

$$anticommutator_sim(v, w) = \frac{|v \cdot w|}{\|v\|_2 \|w\|_2} \quad (4.35)$$

The anticommutator similarity ranges from 0 (orthogonal vectors) to 1 (parallel vectors).

TRANSFER LEARNING WITH ESM2

This chapter presents the mathematical framework for transfer learning using ESM2 protein language models, including the Transformer architecture, embedding generation, LoRA fine-tuning, and feature integration.

5.1 Introduction to Protein Language Models

ESM2 (Evolutionary Scale Modeling) is a protein language model trained on millions of protein sequences [24]. The model uses a Transformer architecture to learn representations that capture evolutionary, structural, and functional information. The key insight of ESM2 is that the patterns of amino acid co-variation across millions of sequences encode information about protein structure and function, similar to how word co-occurrence patterns in natural language encode semantic information.

The ESM2 model used in SGPMAN8.0 is `esm2_t6_8M_UR50D`, which has:

- 6 Transformer layers
- 8 million parameters (smallest ESM2 model, optimized for CPU)
- 320-dimensional embedding space
- Trained on the UR50D dataset (50 million protein sequences)

5.2 Mathematical Framework

Transformer Architecture

The ESM2 model uses a Transformer encoder architecture with the following components:

1. **Input Embedding:** Each residue is embedded using a learned embedding matrix $E \in \mathbb{R}^{20 \times d}$, where $d = 320$ is the embedding dimension.
2. **Positional Encoding:** Sinusoidal positional encodings are added to the input embeddings to capture sequence position information.
3. **Self-Attention:** Multi-head self-attention with $h = 8$ heads:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5.1)$$

4. **Feed-Forward Network:** Two-layer MLP with GELU activation:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (5.2)$$

5. **Layer Normalization:** Applied before each sub-layer for stability.

Embedding Generation

For a protein sequence $S = s_1 s_2 \dots s_L$, the ESM2 embedding is the mean of the per-residue representations:

$$E_{\text{ESM2}}(S) = \frac{1}{L} \sum_{i=1}^L h_i \quad (5.3)$$

where h_i is the hidden representation of residue i from the final layer of the Transformer.

LoRA Fine-Tuning

For efficient fine-tuning, SGPMAIN8.0 uses Low-Rank Adaptation (LoRA) [25]. The key insight of LoRA is that the weight updates during fine-tuning can be represented as a low-rank matrix, dramatically reducing the number of trainable parameters.

The update to a weight matrix $W \in \mathbb{R}^{d \times k}$ is:

$$W = W_0 + \Delta W = W_0 + BA \quad (5.4)$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$. The rank r is typically 8, providing efficient adaptation with minimal parameters.

The LoRA configuration in SGPMAIN8.0 uses:

- Rank $r = 8$
- Alpha $\alpha = 16$
- Dropout $p = 0.1$
- Applied to query and value projection matrices

This reduces the number of trainable parameters from millions to thousands, enabling efficient fine-tuning on domain-specific data.

5.3 Feature Integration

The complete feature vector for a protein is the concatenation of:

$$F(S) = [\text{PIM}(S) \oplus \text{Phys}(S) \oplus E_{\text{ESM2}}(S) \oplus \text{Dis}(S) \oplus \text{Grass}(S)] \quad (5.5)$$

where $\oplus$ denotes concatenation.

The dimensions are:

- PIM: 16 dimensions
- Physicochemical: 21 dimensions (amino acid composition, net charge, hydrophobicity, entropy)
- ESM2: 320 dimensions
- Disorder: 50 dimensions (metapredict scores for first 50 residues)
- Grassmann metrics: 3 dimensions (entropy, Gini coefficient, structural complexity)

Total: 410 dimensions.

5.4 Mathematical Properties of ESM2 Embeddings

ESM2 embeddings have several important mathematical properties:

1. **Continuity:** Small changes in the sequence result in small changes in the embedding.
2. **Information Richness:** The embeddings capture evolutionary, structural, and functional information.
3. **Dimensionality:** The 320-dimensional embedding space provides a compact yet informative representation.
4. **Transferability:** Embeddings trained on general protein sequences can be fine-tuned for specific tasks.

MULTI-OBJECTIVE PEPTIDE DESIGN

This chapter presents the mathematical framework for multi-objective peptide design, including Gaussian Process regression, Bayesian optimization, UCB acquisition, Pareto front optimization, and active learning.

6.1 Gaussian Process for Bayesian Optimization

The peptide generator uses Gaussian Process regression for Bayesian optimization [26]. The GP is defined by:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')) \quad (6.1)$$

Kernel Function

The kernel used is a combination of Matérn and White kernels:

$$k(x, x') = \text{Matern}(\nu = 2.5) + \text{WhiteKernel}(\text{noise} = 10^{-3}) \quad (6.2)$$

The Matérn kernel with $\nu = 2.5$ is defined as:

$$\text{Matern}_{\nu=2.5}(r) = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right) \quad (6.3)$$

where $r = \|x - x'\|$ and ℓ is the length scale parameter.

Upper Confidence Bound (UCB) Acquisition

The acquisition function guides the search for optimal peptides:

$$\alpha_{\text{UCB}}(x) = \mu(x) + \kappa\sigma(x) \quad (6.4)$$

where $\mu(x)$ is the predicted mean, $\sigma(x)$ is the predicted standard deviation, and κ is the exploration-exploitation trade-off parameter (default $\kappa = 2.0$).

6.2 Pareto Front Optimization

The Pareto front contains non-dominated solutions:

$$\mathcal{P} = \{p \in \mathcal{C} \mid \nexists q \in \mathcal{C} : q \succ p\} \quad (6.5)$$

where $q \succ p$ means q dominates p in all objectives. A solution q dominates p if:

$$\forall i : f_i(q) \geq f_i(p) \quad \text{and} \quad \exists j : f_j(q) > f_j(p) \quad (6.6)$$

The objectives optimized are:

1. **Antiviral Activity:** Maximize (higher is better)
2. **Selectivity Index:** Maximize (higher is better)
3. **Stability:** Maximize (higher is better)
4. **Toxicity:** Minimize (lower is better)

6.3 Peptide Mutation Strategy

The mutation strategy identifies critical positions based on PIM values:

$$\text{CriticalPositions}(v) = \arg \max_i |v_i - \text{target}_i| \quad (6.7)$$

Amino acid options for each position are selected based on desired polarity:

$$\text{AAOptions}(p) = \{a \in \mathcal{A} \mid \phi(a) = \text{desired_polarity}(p)\} \quad (6.8)$$

6.4 Active Learning Loop

The optimization process follows an active learning loop:

1. Initialize with a set of candidate peptides
2. Evaluate candidates using the multi-objective predictor
3. Train the GP model on the evaluated candidates
4. Use UCB acquisition to select the next candidate
5. Mutate the selected candidate to generate new sequences
6. Repeat steps 2-5 for N iterations
7. Return the Pareto front of optimal candidates

DRUG-LIKENESS FILTERS

This chapter presents the drug-likeness filters used to evaluate peptide candidates, including Lipinski-modified rules, Veber rules, toxicity filters, stability filters, synthesis feasibility, and the total drug-likeness score.

7.1 Lipinski-Modified Rules

For peptides, the Lipinski rules are modified [20]:

$$\text{Lipinski}(S) = 1 - 0.3 \cdot \mathbb{I}\{|S| < 5\} - 0.2 \cdot \mathbb{I}\{|S| > 30\} - 0.2 \cdot \mathbb{I}\{\text{Hydrophobic} > 0.6\} \quad (7.1)$$

where $\mathbb{I}\{\cdot\}$ is the indicator function, and $|S|$ is the sequence length.

7.2 Veber Rules

Veber rules for peptides [21]:

$$\text{Veber}(S) = 1 - 0.3 \cdot \mathbb{I}\{\text{Flexible} < 0.1\} - 0.2 \cdot \mathbb{I}\{\text{Polar} > 0.7\} \quad (7.2)$$

where Flexible is the fraction of flexible residues (G, S, P, N), and Polar is the fraction of polar residues (N, Q, S, T, Y).

7.3 Toxicity Filters

$$\text{Toxicity}(S) = 1 - 0.3 \cdot \mathbb{I}\{\text{CC50} < 10\} - 0.2 \cdot \mathbb{I}\{\text{Hemolysis} > 50\%\} \quad (7.3)$$

where CC50 is the half-maximal cytotoxic concentration in μM , and Hemolysis is the percentage of hemolytic activity.

7.4 Stability Filters

$$\text{Stability}(S) = 1 - 0.3 \cdot \mathbb{I}\{\text{Half-life} < 4\} - 0.05 \cdot \text{ProteaseSites}(S) \quad (7.4)$$

where Half-life is the predicted plasma half-life in hours, and ProteaseSites is the number of protease recognition sites.

7.5 Synthesis Feasibility

$$\text{Synthesis}(S) = 1 - 0.1 \cdot \mathbb{I}\{\text{Difficult motifs}\} - 0.1 \cdot \mathbb{I}\{\text{Cysteine present}\} \quad (7.5)$$

where Difficult motifs include GGG, PPP, SSS, and AAA.

7.6 Total Drug-Likeness Score

The total drug-likeness score is a weighted sum:

$$\mathcal{D}(S) = \sum_{f \in \mathcal{F}} w_f \cdot \text{Filter}_f(S) \quad (7.6)$$

where $\mathcal{F}$ is the set of filters and w_f are their weights:

Table 7.1: Drug-likeness filter weights

Filter	Weight
Lipinski_modified	0.25
Veber_rules	0.15
Toxicity_filters	0.30
Stability_filters	0.15
Synthesis_feasibility	0.15

7.7 Recommendation Criteria

Based on the total drug-likeness score, the system provides recommendations:

Table 7.2: Recommendations based on drug-likeness score

Score	Recommendation
> 0.8	[OK] CANDIDATO PROMETEDOR - Proceder a validación experimental
0.6 – 0.8	[WARN] CANDIDATO MODERADO - Optimizar con mutaciones dirigidas
< 0.6	[NO] CANDIDATO RECHAZADO - Buscar nuevas secuencias

GRASSMANN MULTILEVEL

This chapter presents the Grassmann multilevel analysis framework, which provides a hierarchical characterization of geometric relationships between PIM vectors at multiple scales.

8.1 grassmann_multilevel_distance

The multilevel distance measures the distance between two vectors in the Grassmann manifold at a given level k .

Definition 8.1.1 (Grassmann Multilevel Distance). *For two PIM vectors $v, w \in \mathbb{R}^{16}$ and a level $k \in \{1, 2, 3\}$, the Grassmann multilevel distance is:*

$$d_k(v, w) = \frac{1}{\sqrt{2k}} \|P_k(v) - P_k(w)\|_F \quad (8.1)$$

where $P_k(v)$ is the projection matrix onto the k -dimensional subspace spanned by the Hankel matrix of v :

$$X_v = \begin{bmatrix} v_1 & v_2 & \cdots & v_k \\ v_2 & v_3 & \cdots & v_{k+1} \\ \vdots & \vdots & \ddots & \vdots \\ v_{16-k+1} & v_{16-k+2} & \cdots & v_{16} \end{bmatrix} \quad (8.2)$$

and $P_k(v) = U_k U_k^T$ where U_k contains the first k left singular vectors of X_v .
The distance ranges from 0 (identical subspaces) to $\sqrt{2}$ (orthogonal subspaces).

8.2 grassmann_multilevel_similarity

The multilevel similarity is derived from the distance:

$$s_k(v, w) = \max(0, 1 - d_k(v, w)) \quad (8.3)$$

8.3 grassmann_projection_asymmetry

The projection asymmetry measures the directional dependence of the projection between two subspaces.

Definition 8.3.1 (Grassmann Projection Asymmetry). *For two PIM vectors $v, w \in \mathbb{R}^{16}$ and a level k , the projection asymmetry is:*

$$d_{12} = 1 - \frac{\|P_k(v)w\|}{\|w\|}, \quad d_{21} = 1 - \frac{\|P_k(w)v\|}{\|v\|} \quad (8.4)$$

$$A_k(v, w) = |d_{12} - d_{21}| \quad (8.5)$$

where $P_k(v)$ is the projection matrix onto the k -dimensional subspace of v .

The asymmetry ranges from 0 (symmetric projection) to 1 (completely asymmetric).

8.4 grassmann_sectional_curvature

The sectional curvature measures the curvature of the Grassmann manifold at a given triple of points.

Definition 8.4.1 (Grassmann Sectional Curvature). *For three PIM vectors $v_1, v_2, v_3 \in \mathbb{R}^{16}$ and a level k , the sectional curvature is:*

$$d_{12} = d_k(v_1, v_2), \quad d_{23} = d_k(v_2, v_3), \quad d_{13} = d_k(v_1, v_3) \quad (8.6)$$

$$s = \frac{d_{12} + d_{23} + d_{13}}{2} \quad (8.7)$$

$$Area(\Delta) = \sqrt{s(s - d_{12})(s - d_{23})(s - d_{13})} \quad (8.8)$$

$$\kappa(v_1, v_2, v_3) = \frac{1}{Area(\Delta) + \epsilon} \quad (8.9)$$

Higher curvature indicates more curved geometry.

8.5 grassmann_sectional_curvature_sampled

When the number of vectors is large, the curvature is sampled:

$$\bar{\kappa} = \frac{1}{N} \sum_{i=1}^N \kappa(v_i, v_j, v_k) \quad (8.10)$$

where N is the number of random triples sampled (default: 30).

8.6 grassmann_volume

The volume measures the spread of points in the Grassmann manifold.

Definition 8.6.1 (Grassmann Volume). *For a set of vectors $\{v_1, \dots, v_n\}$ and a level k , the Grassmann volume is:*

$$V = \frac{\text{mean distance}}{\text{max distance}} \quad (8.11)$$

where:

$$\text{mean distance} = \frac{2}{n(n-1)} \sum_{i < j} d_k(v_i, v_j) \quad (8.12)$$

$$\text{max distance} = \max_{i < j} d_k(v_i, v_j) \quad (8.13)$$

The volume ranges from 0 (all points collapsed) to 1 (well-distributed).

8.7 grassmann_cycles

The cycles detect topological structures in the Grassmann manifold.

Definition 8.7.1 (Grassmann Cycles). *For a set of vectors $\{v_1, \dots, v_n\}$, a level k , and a threshold θ , the cycles are:*

$$\mathcal{C} = \{[i, j, k] \mid s_k(v_i, v_j) > \theta, s_k(v_j, v_k) > \theta, s_k(v_k, v_i) > \theta\} \quad (8.14)$$

Each cycle $[i, j, k]$ represents a triangle in the Grassmann manifold where all pairwise similarities exceed the threshold.

8.8 grassmann_karcher_mean

The Karcher mean is the Riemannian center of mass in the Grassmann manifold.

Definition 8.8.1 (Grassmann Karcher Mean). *For a set of vectors $\{v_1, \dots, v_n\}$ and a level k , the Karcher mean is:*

$$\bar{v} = \arg \min_v \sum_{i=1}^n d_k(v, v_i)^2 \quad (8.15)$$

The optimization is performed iteratively using gradient descent on the Grassmann manifold.

8.9 grassmann_svd_similarity

The SVD similarity compares two subspaces using principal angles.

Definition 8.9.1 (Grassmann SVD Similarity). *For two PIM vectors $v, w \in \mathbb{R}^{16}$ and a level k , the SVD similarity is:*

$$M = U_k(v)^T U_k(w) \tag{8.16}$$

$$\sigma = \text{svd}(M) \tag{8.17}$$

$$\text{principal_angles} = \arccos(\sigma) \tag{8.18}$$

$$\text{svd_sim}(v, w, k) = \text{mean}(\cos(\text{principal_angles})) \tag{8.19}$$

The SVD similarity ranges from 0 (orthogonal subspaces) to 1 (identical subspaces).

THE PROTEIN INTRINSIC DISORDER PREDICTION (PIDP) MODULE

This chapter presents the Protein Intrinsic Disorder Prediction (PIDP) module, which integrates metapredict and AIUPred for robust disorder prediction.

9.1 Motivation and Biological Significance

Protein intrinsic disorder is a fundamental property that has been increasingly recognized as crucial for understanding protein function. Intrinsically disordered proteins and regions (IDPs/IDRs) lack stable three-dimensional structure under physiological conditions yet perform essential functions such as molecular recognition, signaling, and regulation. The importance of disorder in viral proteins is particularly noteworthy, as many viral surface proteins contain disordered regions that facilitate immune evasion, receptor binding, and membrane fusion.

The PIDP module addresses the critical need for accurate disorder prediction in the SGP-MAIN8.0 workflow. By integrating two complementary disorder prediction methods—metapredict and AIUPred—the module provides a robust assessment of protein disorder that can be integrated with the polarity and chemical profiling analysis.

9.2 metapredict: Machine Learning-Based Disorder Prediction

metapredict [22] employs a machine learning approach that combines multiple sequence features to predict disorder propensity. The algorithm processes an input sequence through the following stages:

1. **Sequence Encoding:** The amino acid sequence is encoded using one-hot encoding, which represents each residue as a binary vector of length 20.
2. **Evolutionary Information:** A position-specific scoring matrix (PSSM) derived from sequence homology searches is incorporated.
3. **Secondary Structure:** Secondary structure predictions from a neural network are included as additional features.
4. **Random Forest Classification:** A Random Forest classifier with 500 decision trees integrates all features to produce a per-residue disorder score.

The output of metapredict is a per-residue disorder score in the range from 0 to 1, where scores above 0.5 typically indicate disorder. The model was trained on a curated dataset

of proteins with experimentally validated disorder, achieving an area under the ROC curve (AUC) of 0.89.

9.3 AIUPred: Biophysical Model-Based Disorder Prediction

AIUPred [23] takes a fundamentally different approach, employing a biophysical model based on amino acid composition and sequence context. The algorithm is rooted in the observation that disorder propensity is largely determined by the balance of charged, polar, and hydrophobic residues.

The AIUPred algorithm models disorder as a combination of intrinsic residue propensities and context-dependent effects. Each residue contributes a position-specific bias term, and neighboring residues contribute context-specific weights. This approach captures both the local amino acid preferences for disorder and the influence of the surrounding sequence environment.

A key feature of AIUPred is its ability to predict redox sensitivity profiles. The algorithm can generate two separate disorder profiles: one for reducing conditions (redox-plus) and one for oxidizing conditions (redox-minus). The redox sensitivity is then defined as the difference between these two profiles, averaged over the sequence.

9.4 Integration with the SGPMAN8.0 Workflow

The PIDP module is seamlessly integrated into the SGPMAN8.0 workflow through the PIDPProfiler class. The integration occurs after the PIM and chemical profiling stages, allowing for a complete characterization that includes disorder information. The key design principles are:

1. **Compatibility:** The PIDP module operates on the same sequence data that is used for PIM calculation, ensuring consistency between analyses.
2. **Complementarity:** Disorder analysis provides information that is orthogonal to polarity and chemical profiling, filling gaps in structural understanding.
3. **Actionability:** PIDP results are presented in a format that directly supports experimental design and interpretation.

9.5 PIDP Output Interpretation

The PIDP module produces several outputs:

Table 9.1: PIDP output interpretation

Disorder Level	Interpretation
0-10%	Predominantly ordered protein. Suitable for crystallization.
10-30%	Flexible regions present. May be important for function.
30-50%	Significant disorder. Likely IDP or has large IDRs.
>50%	Highly disordered. Consider IDP-specific methods.

THE CHEMICALPROFILER MODULE

This chapter presents the ChemicalProfiler module, which derives 54 chemical properties from the PIM vector organized into 14 categories. These properties connect the abstract mathematics to practical biochemistry.

10.1 Charge Properties (7 properties)

The charge properties capture the electrostatic characteristics of the polarity profile:

$$\text{net_charge} = v_1 + v_4 - v_0 - v_5 \quad (10.1)$$

$$\text{charge_density} = v_0 + v_1 + v_4 + v_5 \quad (10.2)$$

$$\text{charge_balance} = \frac{v_1 + v_4}{v_0 + v_5 + \epsilon} \quad (10.3)$$

Properties: net_charge, charge_density, charge_balance, positive_positive, positive_negative, negative_positive, negative_negative.

10.2 pKa/pI Properties (5 properties)

The pKa and pI properties characterize the acid-base behavior of the protein:

$$\text{pI} = 6.0 - 0.5 \cdot \text{net_charge} \quad (10.4)$$

Properties: pKa_H54, pKa_E64, pKa_D66, pKa_H516, pKa_K68, pKa_C512, pI.

10.3 Electrostatic Properties (4 properties)

The electrostatic properties characterize the surface charge distribution:

$$\text{positive_charge_fraction} = \frac{v_0 + v_2 + v_8}{\text{total}} \quad (10.5)$$

$$\text{negative_charge_fraction} = \frac{v_5 + v_6 + v_9}{\text{total}} \quad (10.6)$$

$$\text{surface_charge_density} = \frac{v_0 + v_1 + v_4 + v_5}{\text{total}} \quad (10.7)$$

Properties: positive_charge_fraction, negative_charge_fraction, surface_charge_density, electrostatic_potential.

10.4 Hydrophobicity Properties (4 properties)

The hydrophobicity properties characterize the hydrophobic character of the protein:

$$\text{GRAVY} = 4.5 \cdot \frac{v_{15} + v_{14} + v_{11} - v_{10} - v_2 - v_6 - v_8 - v_9}{\text{total}} \quad (10.8)$$

Properties: GRAVY, hydrophobicity_score, num_hydrophobic_components, hydrophobic_peaks.

10.5 Hydrophobic Patches (3 properties)

Hydrophobic patches are contiguous regions with significant hydrophobic character:

$$\text{num_patches} = \text{number of contiguous regions with } v_i > 0.01 \quad (10.9)$$

Properties: num_patches, patch_scores, hydrophobic_component_fraction.

10.6 Glycosylation Sites (2 properties)

Glycosylation sites are identified using regular expressions:

- N-glycosylation: `N[^P] [ST]`
- O-glycosylation: `[ST]`

Properties: n_glycosylation_count, o_glycosylation_count.

10.7 PTM Sites (3 properties)

Phosphorylation motifs:

- PKA: [RK] [ST] [ˆP]
- PKC: [ST] [ˆP] [KR]
- CK2: [ST] [ˆP] [DE]
- Tyr: Y[ˆP]

Properties: phosphorylation_count, acetylation_count, ubiquitination_count.

10.8 Solubility Properties (3 properties)

The solubility properties characterize the protein's behavior in aqueous solution:

$$\text{solubility} = 10 + 10 \cdot \frac{v_{10} + v_1 + v_4}{\text{total}} - 5 \cdot \frac{v_{15} + v_{14} + v_{11}}{\text{total}} \quad (10.10)$$

Properties: solubility_mg_ml, aggregation_score, aggregation_regions.

10.9 Stability Properties (3 properties)

The stability properties characterize the protein's conformational stability:

$$\Delta G = -4.0 - 5.0 \cdot \frac{v_{10} + v_1 + v_4}{v_{10} + v_1 + v_4 + v_{15} + \epsilon} \quad (10.11)$$

$$T_m = 30 + 20 \cdot \frac{v_{10} + v_1 + v_4}{v_{10} + v_1 + v_4 + v_{15} + \epsilon} \quad (10.12)$$

Properties: delta_g, tm, stability_score.

10.10 Hotspots (3 properties)

Hotspots are identified in three regions:

1. Charge-rich region (CR2-like): score based on $v_1 + v_4 + v_0 + v_5$
2. Hydrophobic patch (fusion loop-like): score based on $v_{15} + v_{14} + v_{11}$
3. Mixed polar/hydrophobic: score based on $v_{11} + v_{14} + v_2 + v_6$

Properties: hotspot_1, hotspot_2, hotspot_3.

10.11 Reactivity Properties (3 properties)

The reactivity properties characterize the accessibility of reactive residues:

$$\text{cysteine_access} = \min(1, 2 \cdot (v_0 + v_1 + v_2 + v_8 + v_9)) \quad (10.13)$$

Properties: cysteine_accessibility, lysine_accessibility, tyrosine_accessibility.

10.12 Metal Binding Properties (2 properties)

The metal binding properties predict affinity for divalent cations:

$$\text{Ca}^{2+} \text{ score} = \min(1, 1.5 \cdot (v_1 + v_4 + v_6 + v_7 + v_9 + v_{13})) \quad (10.14)$$

$$\text{Zn}^{2+} \text{ score} = \min(1, 1.2 \cdot (v_0 + v_2 + v_3 + v_8 + v_{10} + v_{12})) \quad (10.15)$$

Properties: calcium_binding_score, zinc_binding_score.

10.13 Membrane Permeability Properties (2 properties)

The membrane permeability properties characterize interaction with lipid bilayers:

$$\text{permeability} = 0.5 + 0.3 \cdot v_{15} - 0.2 \cdot (v_0 + v_1 + v_4 + v_5) \quad (10.16)$$

Properties: permeability_score, membrane_affinity.

10.14 Lipid Binding Properties (4 properties)

The lipid binding properties predict affinity for different lipid types:

$$\text{PS score} = \min(1, 1.5 \cdot (0.6 \cdot v_{15} + 0.4 \cdot (v_1 + v_4))) \quad (10.17)$$

$$\text{Cholesterol score} = \min(1, 1.5 \cdot 0.8 \cdot v_{15}) \quad (10.18)$$

$$\text{Ganglioside score} = \min(1, 1.5 \cdot (0.3 \cdot v_{15} + 0.7 \cdot v_{10})) \quad (10.19)$$

Properties: phosphatidylserine_score, cholesterol_score, ganglioside_score, best_lipid.

10.15 Buffer Stability Properties (4 properties)

The buffer stability properties provide recommendations for experimental conditions:

$$\text{optimal_pH} = \max(5.0, \min(8.0, 7.0 - 0.5 \cdot \text{net_charge})) \quad (10.20)$$

$$\text{salt_tolerance} = 150 + 100 \cdot (1 - v_1 - v_4 - v_0 - v_5) \quad (10.21)$$

$$\text{glycerol} = 5 + 10 \cdot (v_{15} + v_{14} + v_{11}) \quad (10.22)$$

Properties: optimal_pH, salt_tolerance_mM, glycerol_percent, buffer_recommendation, storage_temperature.

CLASES DEL SISTEMA - PARTE 1: CLASES BASE Y ALMACENAMIENTO

This chapter presents the base classes of the SGPMAN8.0 system, including DiskCache, SVDCache, ProcessingTracker, OnlineStatistics, ProgressiveSampler, PIMHashIndex, and GroupStatistics.

11.1 DiskCache

The DiskCache class provides persistent storage for PIM vectors and ESM2 embeddings. This avoids redundant computation when analyzing the same sequences multiple times.

Table 11.1: DiskCache methods

Method	Description
get_key(sequence)	Generates a hash key for the sequence
get_pim(sequence)	Retrieves a PIM vector from cache
save_pim(sequence, pim)	Saves a PIM vector to cache
get_esm_embedding(sequence)	Retrieves an ESM2 embedding from cache
save_esm_embedding(sequence, embedding)	Saves an ESM2 embedding to cache
get_stats()	Returns cache statistics (hits, misses, hit rate)

11.2 SVDCache

The SVDCache class caches singular value decompositions used in Grassmann multilevel analysis.

Table 11.2: SVDCache methods

Method	Description
get_svd(vector, k)	Retrieves or computes SVD for a vector at level k
get_stats()	Returns cache statistics (hits, misses, hit rate, cache size)

11.3 ProcessingTracker

The ProcessingTracker class monitors the progress of file processing.

Table 11.3: ProcessingTracker methods

Method	Description
update(group_name, is_valid, bytes_read)	Updates processing statistics
get_report()	Returns a complete processing report
print_progress(group_name, force)	Prints current progress
print_summary()	Prints a global processing summary

11.4 OnlineStatistics

The OnlineStatistics class computes statistics incrementally without storing all vectors.

Table 11.4: OnlineStatistics methods

Method	Description
update(x)	Updates statistics with a new vector
get_covariance()	Returns the covariance matrix
get_mean()	Returns the mean vector
get_std()	Returns the standard deviation vector

11.5 ProgressiveSampler

The ProgressiveSampler class performs reservoir sampling to maintain a representative subset of vectors.

Table 11.5: ProgressiveSampler methods

Method	Description
add(vector, header)	Adds a vector to the sample using reservoir sampling
get_samples()	Returns the sampled vectors
get_headers()	Returns the sampled headers
size()	Returns the number of samples

11.6 PIMHashIndex

The PIMHashIndex class provides locality-sensitive hashing for fast similarity search.

Table 11.6: PIMHashIndex methods

Method	Description
<code>add_protein(protein_id, group, vector)</code>	Adds a protein to the index
<code>search(vector)</code>	Searches for similar proteins
<code>build_from_samples(samples)</code>	Builds the index from sample data

11.7 GroupStatistics

The GroupStatistics class stores comprehensive statistics for each protein group.

Table 11.7: GroupStatistics fields

Field	Description
<code>name</code>	Group name
<code>n_samples</code>	Number of valid PIM vectors
<code>centroid</code>	Mean PIM vector
<code>covariance</code>	Covariance matrix
<code>inv_covariance</code>	Inverse covariance matrix
<code>std_dev</code>	Standard deviation vector
<code>wedge_self_similarity</code>	Internal cohesion of the group
<code>adaptive_threshold</code>	5th percentile of intra-group similarities
<code>clifford_signature</code>	Clifford signature of the centroid
<code>subspace_projections</code>	Projections onto predefined subspaces
<code>entropy</code>	Shannon entropy of the centroid
<code>gini</code>	Gini coefficient of the centroid
<code>complexity</code>	Structural complexity of the centroid
<code>modularity</code>	Functional modularity of the centroid
<code>morans_i</code>	Moran's I of the centroid
<code>grassmann_multilevel</code>	Multilevel Grassmann distances
<code>grassmann_asymmetry</code>	Projection asymmetry values
<code>grassmann_curvature</code>	Sectional curvature
<code>grassmann_volume</code>	Grassmann volume
<code>grassmann_cycles</code>	Topological cycles
<code>grassmann_karcher_centroid</code>	Karcher mean
<code>grassmann_svd_angles</code>	SVD principal angles

CLASES DEL SISTEMA - PARTE 2: CLASES PRINCIPALES

This chapter presents the main classes of the SGPMAIN8.0 system, including GrassmannPIM, AdvancedGroupAnalyzer, EnhancedFeatureExtractor, MultiObjectivePredictor, Drug-LikenessFilter, PeptideGenerator, and PeptideDesignEngine.

12.1 GrassmannPIM

The GrassmannPIM class is the mathematical engine of the system. It implements all the geometric algebra operations described in previous chapters.

Table 12.1: GrassmannPIM methods

Method	Description
wedge_product(v, w, with_ci)	Calculates wedge product with optional conf
wedge_product_oriented(v, w)	Calculates oriented wedge product with bive
interior_product_magnitude(v, subspace)	Calculates magnitude of interior product
specular_reflection(v)	Applies specular reflection to vector
is_specular_reflection(v1, v2, threshold)	Checks if two vectors are related by reflection
all_rotor_angles(v, w)	Calculates all rotor angles
reflection_analysis(v, w)	Complete reflection analysis
clifford_signature(v)	Calculates Clifford signature
dot_product_metric(v, w)	Metric dot product
norm_metric(v)	Metric norm
similarity_metric(v, w)	Metric similarity
metric_signature_info()	Returns metric signature information
commutator_norm(v, w)	Calculates commutator norm
anticommutator_similarity(v, w)	Calculates anticommutator similarity
geometric_product_full(v, w)	Calculates full geometric product
hodge_dual(v)	Calculates Hodge dual
hodge_complementarity(v1, v2)	Calculates Hodge complementarity
grassmann_distance(v1, v2)	Calculates Grassmann distance
grassmann_geodesic(v1, v2, n_steps)	Calculates Grassmann geodesic
grassmann_projection_distance(v1, v2)	Calculates projection distance
grassmann_fubini_study(v1, v2)	Calculates Fubini-Study metric
grassmann_ricci_curvature(v1, v2)	Calculates Ricci curvature
karhunen_loeve_decomposition(vectors, n_components)	Performs KL decomposition
compute_enhanced_metrics(v1, v2)	Computes all enhanced metrics
multilevel_distance(v1, v2, k)	Calculates multilevel distance
multilevel_similarity(v1, v2, k)	Calculates multilevel similarity
projection_asymmetry(v1, v2, k)	Calculates projection asymmetry
sectional_curvature(v1, v2, v3, k)	Calculates sectional curvature
sectional_curvature_sampled(vectors, k, n_samples)	Sampled sectional curvature
volume(vectors, k)	Calculates Grassmann volume
cycles(vectors, k, threshold)	Detects topological cycles
karcher_mean(vectors, k)	Calculates Karcher mean
svd_similarity(v1, v2, k)	Calculates SVD similarity
get_cache_stats()	Returns cache statistics

12.2 AdvancedGroupAnalyzer

The AdvancedGroupAnalyzer class is the core of the analysis pipeline. It manages loading, processing, and comparing protein groups.

Table 12.2: AdvancedGroupAnalyzer methods

Method	Description
load_fasta_file(filepath, group_name, verbose)	Loads a FASTA file
load_fasta_unlimited(filepath, group_name, verbose)	Loads a FASTA file without size limit
set_sample_size(size)	Sets the maximum number of proteins to store
compare_group_to_all(target_group)	Compares target group to all others
get_top_individual_proteins(target_group, top_n)	Gets top N individual proteins
cross_group_similarity_matrix()	Computes cross-group similarity matrix
build_hash_index()	Builds LSH hash index
print_processing_summary()	Prints processing summary
generate_full_report(target_group, results_dir)	Generates complete report

12.3 EnhancedFeatureExtractor

The EnhancedFeatureExtractor class combines PIM, physicochemical, ESM2, disorder, and Grassmann features.

Table 12.3: EnhancedFeatureExtractor methods

Method	Description
load_esm2(model_name)	Loads the ESM2 model
get_esm_embedding(sequence, use_cache)	Gets ESM2 embedding for a sequence
extract_all_features(sequence, pim)	Extracts all features from a sequence
get_feature_vector(features)	Concatenates all features into a single vector
get_feature_dimension()	Returns total feature dimension (410)

12.4 MultiObjectivePredictor

The MultiObjectivePredictor class predicts multiple properties of antiviral peptides using ensemble models.

Table 12.4: MultiObjectivePredictor methods

Method	Description
train_models(X, y_dict)	Trains models for each target
predict(X)	Predicts all properties for a feature vector
predict_peptide(sequence, feature_extractor)	Predicts properties for a peptide sequence

The available targets are:

- antiviral_activity: IC50/EC50 (normalized 0-1)

- cytotoxicity: CC50 (μM)
- stability: Plasma half-life (hours)
- selectivity_index: CC50/IC50 (ratio)
- hemolytic_activity: Hemolysis (%)
- immunogenicity: Immunogenic potential (normalized 0-1)

12.5 DrugLikenessFilter

The DrugLikenessFilter class evaluates peptide candidates against multiple drug-likeness criteria.

Table 12.5: DrugLikenessFilter methods

Method	Description
evaluate(sequence, predictions)	Evaluates a peptide against all filters

The filters implemented are:

1. Lipinski_modified: Modified Lipinski rules for peptides
2. Veber_rules: Flexibility and polarity
3. Toxicity_filters: Cytotoxicity and hemolysis
4. Stability_filters: Half-life and protease resistance
5. Synthesis_feasibility: Synthesis difficulty

12.6 PeptideGenerator

The PeptideGenerator class uses Gaussian Process optimization to generate optimized peptides.

Table 12.6: PeptideGenerator methods

Method	Description
setup_gp(X_train, y_train)	Configures Gaussian Process model
optimize_sequence(target_sequence, n_iterations)	Optimizes a sequence using GP

The optimization process includes:

1. Critical position identification based on PIM differences

2. Amino acid selection based on desired polarity
3. Mutation of selected positions
4. Upper Confidence Bound (UCB) acquisition
5. Pareto front maintenance

12.7 PeptideDesignEngine

The PeptideDesignEngine class is the complete peptide design engine that integrates all components.

Table 12.7: PeptideDesignEngine methods

Method	Description
<code>train(X_train, y_dict)</code>	Trains all models
<code>design_peptide(target_sequence, n_iterations, n_candidates)</code>	Designs optimized peptides

The design process includes:

1. Feature extraction using ESM2 and PIM
2. Multi-objective prediction
3. Gaussian Process optimization
4. Drug-likeness filtering
5. Pareto front optimization
6. Candidate ranking and selection

CHEMBLMAPPER

The ChEMBLMapper class maps UniProt proteins to ChEMBL targets using the chembl_uniprot.txt mapping file.

Table 13.1: ChEMBLMapper methods

Method	Description
get_chembl_id(uniprot_id)	Gets ChEMBL ID for a UniProt ID
get_uniprot_id(chembl_id)	Gets UniProt ID for a ChEMBL ID
search_by_name(name)	Searches for proteins by name

13.1 Mapping File Format

The mapping file follows the format from ChEMBL:

```
# chembl_37 target list, 01/05/2026
P21266 CHEMBL2242      Glutathione S-transferase Mu 3  SINGLE PROTEIN
P27658 CHEMBL3314      Collagen alpha-1(VIII) chain  SINGLE PROTEIN
```


CHAPTER 14

APDLOADER

The APDLoader class loads antimicrobial peptides from the Antimicrobial Peptide Database (APD).

Table 14.1: APDLoader methods

Method	Description
<code>get_all_peptides()</code>	Returns all peptides
<code>get_active_peptides(threshold)</code>	Returns peptides with activity above threshold
<code>get_inactive_peptides(threshold)</code>	Returns peptides with activity below threshold

14.1 Activity Estimation

For peptides without experimentally measured activity, the APDLoader estimates activity based on sequence properties:

1. Length: 10-30 amino acids (optimal)
2. Cationic charge: K and R content $> 20\%$
3. Hydrophobicity: A, L, I, V, F, W content $> 30\%$
4. Polarity: N, Q, S, T content $> 15\%$
5. Sequence diversity: Unique amino acids > 5

$$\begin{aligned}
 \text{activity} = & 0.5 + 0.15 \cdot \mathbb{I}\{10 \leq |S| \leq 30\} \\
 & + 0.15 \cdot \mathbb{I}\{\text{cationic} > 0.2\} \\
 & + 0.1 \cdot \mathbb{I}\{\text{hydrophobic} > 0.3\} \\
 & + 0.1 \cdot \mathbb{I}\{\text{polar} > 0.15\} \\
 & + 0.1 \cdot \mathbb{I}\{\text{unique} > 5\}
 \end{aligned} \tag{14.1}$$

THERAPEUTICPROFILER

The TherapeuticProfiler class designs synthetic competitor peptides based on the target protein profile.

Table 15.1: TherapeuticProfiler methods

Method	Description
<code>predict_activity(peptide_sequence)</code>	Predicts antiviral activity of a peptide
<code>generate_therapeutic_profile()</code>	Generates complete therapeutic profile
<code>print_profile(profile)</code>	Prints the therapeutic profile

15.1 Target Identification

The therapeutic profile identifies the most similar membrane target:

1. Search membrane groups: MEMBRANE, REVIEWED_HUMAN, UNREVIEWED_HUMAN
2. Compute wedge similarity between target and each group
3. Apply score: $\text{score} = \text{similarity} * (0.8 + 0.2 * (1 \text{ if ChEMBL ID available else } 0))$
4. Select the group with the highest score

15.2 Peptide Design

The peptide is designed based on critical polarity differences:

1. Compute difference: $d = v_{\text{target}} - v_{\text{host}}$
2. Identify critical indices: $\arg \max_i |d_i|$ (top 5)
3. Map each critical transition to amino acids:

Table 15.2: Transition to amino acid mapping

Transition	Preferred Amino Acids
$P^+ \rightarrow P^-$	K, R, H, D, E
$P^- \rightarrow P^+$	D, E, K, R, H
$N \rightarrow N$	N, Q, S, T, Y
$NP \rightarrow NP$	L, V, I, A, F, W
$P^+ \rightarrow N$	K, R, N, Q, S
$P^- \rightarrow N$	D, E, N, Q, S
$P^+ \rightarrow NP$	K, R, L, V, A
$P^- \rightarrow NP$	D, E, L, V, A

15.3 Activity Prediction Model

The activity prediction model uses a RandomForestRegressor trained on APD peptides:

- 100 estimators
- Max depth: 10
- Features: PIM (16) + length + net charge + hydrophobicity + entropy
- Training: 80% training, 20% testing
- Metrics: R^2 , MSE

15.4 Physicochemical Properties

The TherapeuticProfiler calculates the following properties for designed peptides:

- Net charge: $\sum \text{charges}(aa)$
- Molecular weight: $\sum \text{weights}(aa)$
- Hydrophobicity: mean of Kyte-Doolittle scale
- Isoelectric point: $pI = 6.0 - 0.5 \times \text{net_charge}$
- Solubility: $10 + (1 - |\text{net_charge}|/3) \times 5 - \max(0, \text{hydrophobicity}) \times 2$

15.5 Comparison with Known Inhibitors

The TherapeuticProfiler compares the designed peptide with known inhibitors:

Table 15.3: Known inhibitors for comparison

Inhibitor	IC50 (nM)	Type
Remdesivir	0.08	small_molecule
Favipiravir	0.25	small_molecule
ZMapp	0.015	antibody
REGN-EB3	0.012	antibody
mAb114	0.009	antibody

15.6 Recommendations

The TherapeuticProfiler generates recommendations including:

1. Synthesis method (solid-phase peptide synthesis)
2. Formulation buffer (PBS pH 7.4 or 10% DMSO + PBS)
3. Protecting groups (for N and Q residues)
4. Stabilization strategy (cyclization or PEGylation)
5. Validation assays (SPR/ITC binding assays)
6. Optimization suggestions (if activity score < 0.6)

THE FUNCTION `main()`

This chapter describes the `main()` function, which serves as the entry point of the SGP-MAIN8.0 program and orchestrates the entire analysis pipeline.

16.1 Execution Flow

The `main()` function follows these steps:

1. **Initial Configuration:** Displays system information, configuration parameters, and available features.
2. **Initialize Components:** Creates the GrassmannPIM, AdvancedGroupAnalyzer, and sets up processing parameters.
3. **Load Files:** Iterates through the list of group files and loads each one using `load_fasta_file()`.
4. **Process Summary:** Displays global processing statistics including total sequences, validity rate, and processing speed.
5. **Build Hash Index:** Creates the LSH hash index for fast similarity search.
6. **Create Results Directory:** Creates a timestamped directory for output files.
7. **Identify Target Group:** Determines the target group from `MAIN_GROUP` or uses the first available group.
8. **Generate Full Report:** Calls `generate_full_report()` which produces:
 - Comparison matrix (target vs all groups)
 - Cross-group similarity matrix
 - Top individual proteins
 - Chemical profiles
 - KL decomposition
 - Grassmann multilevel report
 - PIDP analysis
 - Therapeutic profile
9. **Peptide Design:** If a target sequence is available:
 - a) Load APD peptides for training data

- b) Extract features using ESM2 and PIM
 - c) Train MultiObjectivePredictor
 - d) Design optimized peptides using PeptideDesignEngine
 - e) Save design results
10. **Save Results:** Saves all outputs to the results directory.
 11. **Display Summary:** Shows execution time and cache statistics.

16.2 File Loading Order

The files are loaded in the following order:

```
sudan -> zaire -> reston -> bombali -> bundibugyo -> tai ->
lasv -> junv -> macv -> lcmv -> nile1 -> nile2 -> lujo ->
PARTIALLY_FOLDED -> CPP -> NON_CPP -> UNFOLDED ->
REVIEWED_HUMAN -> UNREVIEWED_HUMAN -> senales -> membrana ->
enfermedad -> VIRUS_REVIEWED -> VIRUS_UNREVIEWED ->
REVIEWED_ALL -> UNREVIEWED_ALL
```

16.3 Results Directory Structure

The results directory contains:

```
results_v8_YYYYMMDD_HHMMSS/
+-- comparison_{target}_vs_all.csv
+-- similarity_matrix_groups.csv
+-- top_20_individual_proteins.csv
+-- chemical_profile_{group}.csv          (for each group)
+-- kl_decomposition.json
+-- therapeutic_profile.json
+-- pidp_analysis_{group}.csv            (for each target)
+-- pidp_summary_all_targets.csv
+-- pidp_peptide_analysis.csv
+-- grassmann_multilevel_report.csv
+-- grassmann_cycles_report.csv
+-- designed_peptides.csv
+-- best_candidate.json
+-- design_summary.json
```

16.4 Command-Line Execution

```
# Run the complete analysis
```

```
python3 SGPMIN81.0v.py
```

```
# The results will be saved in:  
# results_v8_YYYYMMDD_HHMMSS/
```

16.5 Error Handling

The `main()` function includes error handling for:

- Missing input files (skips with warning)
- Missing APD or ChEMBL files (continues without them)
- ESM2 model loading failures (continues without ESM2)
- PIDP tool unavailability (continues without PIDP)
- Memory errors (handled by garbage collection)

Part III

Case Studies and Applications

WEST NILE VIRUS CASE STUDY

This chapter presents a comprehensive case study demonstrating the application of SGP-MAIN8.0 to the analysis of West Nile virus (WNV) envelope proteins. The case study covers sequence analysis, PIM profiling, chemical property extraction, hotspot identification, PIDP analysis, and peptide design.

17.1 Introduction to West Nile Virus

West Nile virus (WNV) is a mosquito-borne flavivirus that causes febrile illness and neurological disease in humans. The virus encodes two key envelope proteins that are essential for viral entry and pathogenesis: the small envelope protein M (NILE1) and the envelope glycoprotein E (NILE2). These proteins are the focus of this case study, which demonstrates the complete SGPMAIN8.0 analysis pipeline.

17.2 NILE1: Small Envelope Protein M

The membrane protein M (NILE1) is a small, hydrophobic protein that is cleaved from the precursor prM during viral maturation. It plays a crucial role in membrane fusion and virus entry.

Sequence

```
>P06935_005 West Nile virus; 11082; 216-290; Small envelope protein M
SLTVQTHGESTLANKKGAWLDSTKATRYLVKTESWILRNPGYALVAAVIGWMLGSNTMQR
VVFALLLLLVAPAYS
```

Key Properties

Table 17.1: Key properties of NILE1

Property	Value
Length	75 amino acids
Hydrophobicity	High (contains transmembrane domain)
Function	Membrane fusion, virus entry
Structure	Predominantly α -helical

17.3 NILE2: Envelope Glycoprotein E

The envelope glycoprotein E (NILE2) is the major surface protein of WNV. It is responsible for receptor binding and membrane fusion.

Sequence

```
>P06935_006 West Nile virus; 11082; 291-787; Envelope protein E
FNCLGMSNRDFLEGVSGATWVDLVLEGDSCVTIMSKDKPTIDVKMMNMEAANLADVRSYC
YLASVSDLSTRAACPTMGEAHNEKRADPAFVCKQGVDVRGWGNGCGLFGKGSIDTCAKFA
CTTKATGWIIQKENIKYEVAIFVHGPTTVESHGKIGATQAGRFSITPSAPSYTLKLGEYG
EVTVDCEPRSGIDTSAYYVMSVGEKSFLVHREWFMDLNLWSSAGSTTWRNRETLMEFEE
PHATKQSVVALGSQEGALHQAAGAIPVEFSSNTVKLTSGHLKCRVKMEKLQLKGTTYGV
CSKAFKFARTPADTGHGTVVLELQYTGTDGPCKVPISSVASLNDLTPVGRLVTNPFVSV
ATANSKVLIELEPPFGDSYIVVGRGEQQINHHWHKSGSSIGKAFTTTLRGAQRLAALGDT
AWDFGSVGGVFTSVGKAIHQVFGGAFRSLFGGMSWITQGLLGALLLWMGINARDRSIAMT
FLAVGGVLLFLSVNVHA
```

Key Properties

Table 17.2: Key properties of NILE2

Property	Value
Length	497 amino acids
Structure	Three domains (DI, DII, DIII)
Function	Receptor binding, membrane fusion, immune evasion
Key residues	Fusion loop (98-109), receptor binding domain

17.4 PIM Profiles for WNV Proteins

The PIM profiles for the two WNV proteins reveal their distinct polarity organizations:

Table 17.3: Key PIM components for NILE1 and NILE2

Transition	NILE1 (M)	NILE2 (E)
NP→NP	0.467	0.298
NP→N	0.040	0.067
N→NP	0.027	0.066
P ⁺ →P ⁻	0.000	0.011
P ⁻ →P ⁺	0.000	0.006
NP→P ⁺	0.013	0.031

17.5 Similarity Analysis: NILE1 vs NILE2

The wedge similarity between the two WNV proteins is:

$$\text{WedgeSim}(v_{\text{NILE1}}, v_{\text{NILE2}}) = 0.1264 \quad (17.1)$$

This low similarity value indicates that the two proteins have substantially different polarity profiles, consistent with their different functions: NILE1 is a small membrane protein, while NILE2 is a large surface glycoprotein.

17.6 WNV Chemical Profile Summary

Table 17.4: Selected chemical properties for NILE1 and NILE2

Property	NILE1 (M)	NILE2 (E)
Net Charge	0.267	0.052
Charge Density	0.093	0.085
Charge Balance	0.000	0.808
pI	6.867	5.974
GRAVY	0.369	0.253
N-Glycosylation Sites	5	12
O-Glycosylation Sites	22	64
Phosphorylation Sites	10	32
Solubility (mg/mL)	7.29	12.81
ΔG (kcal/mol)	-4.958	-6.760
T _m (°C)	68.85	41.04
Stability Score	0.780	0.552
Best Lipid Binding	Cholesterol	Cholesterol

17.7 Hotspot Analysis

Three drug-binding hot spots were identified in NILE2:

Table 17.5: Hotspots identified in NILE2

Hotspot Name		Score	Key Residues
1	Charge-rich (CR2-like)	0.104	H54, E64, D66, K68, H71
2	Hydrophobic (fusion loop-like)	0.339	D512, E515, H516, I518, W519, A520
3	Mixed polar/hydrophobic (binding loop-like)	0.268	F134, F136, Y139, D142, K144

17.8 PIDP Analysis for WNV Proteins

Table 17.6: PIDP results for NILE1 and NILE2

Metric	NILE1 (M)	NILE2 (E)
metapredict disorder (0.5)	10.2%	8.15%
metapredict mean score	0.1912	0.1655
AIUPred disorder (0.5)	11.8%	9.69%
AIUPred mean score	0.2045	0.1919
AIUPred redox sensitivity	0.015	0.029

The low disorder percentages confirm that both WNV envelope proteins are predominantly ordered, consistent with their structural roles.

17.9 Peptide Design for WNV

The peptide design engine generated candidate peptides targeting WNV envelope protein E:

Table 17.7: Top designed peptides targeting WNV E protein

Rank	Sequence	Drug Score	Activity	Recommendation
1	FDLGNNKAAAV	0.823	0.853	PROMETEDOR
2	FDLGNNKAATV	0.801	0.842	PROMETEDOR
3	FDLGNNKAAVV	0.789	0.831	MODERADO
4	FDLGNNKAAAV	0.776	0.818	MODERADO
5	FDLGNNKAAAV	0.761	0.805	MODERADO

Best Candidate Rationale

PÉPTIDO DISEÑADO: FDLGNNKAAAV

=====

PERFIL PREDICTIVO:

- Actividad antiviral: 0.853
- Índice de selectividad: 82.4
- Estabilidad: 14.2 horas
- Toxicidad: CC50 = 48.3 μ M
- Hemólisis: 3.8%

DRUG-LIKENESS: 0.823

- Lipinski_modified: Passed
- Veber_rules: Passed
- Toxicity_filters: Passed
- Stability_filters: Passed
- Synthesis_feasibility: Passed

CARACTERÍSTICAS ESTRUCTURALES:

- Longitud: 11 aa
- Composición: F(1), D(1), L(1), G(1), N(2), K(1), A(3), V(1)

REFERENCE DATABASE DESCRIPTION

This appendix provides a comprehensive description of the reference protein databases used in SGPMAIN8.0, including UniProt groups, performance statistics, and available tools.

A.1 UniProt Groups

The reference dataset consists of 26 groups:

Group Code	Description	N
nile1	West Nile virus membrane protein M	1
nile2	West Nile virus envelope glycoprotein E	1
lujo	Lujo virus	1
lasv	Lassa virus	4
junv	Junín virus	3
macv	Machupo virus	3
lcmv	Lymphocytic choriomeningitis virus	3
sudan	Ebola Sudan	1
zaire	Ebola Zaire	1
reston	Ebola Reston	1
bombali	Ebola Bombali	1
bundibugyo	Ebola Bundibugyo	1
tai	Ebola Tai Forest	1
CPP	Cell-Penetrating Peptides	100
NON_CPP	Non-Cell-Penetrating Peptides	71
UNFOLDED	Intrinsically Disordered Proteins	106
PARTIALLY_FOLDED	Partially Folded Proteins	153
REVIEWED_HUMAN	Reviewed Human Proteome	20,416
UNREVIEWED_HUMAN	Unreviewed Human Proteome	127,090
VIRUS_REVIEWED	Reviewed Virus Proteome	17,517
VIRUS_UNREVIEWED	Unreviewed Virus Proteome	1,087,163
REVIEWED_ALL	Reviewed All Proteome	575,503
UNREVIEWED_ALL	Unreviewed All Proteome	149,234,636
SIGNALS	Signaling Proteins	47
MEMBRANE	Membrane Proteins	165
DISEASE	Disease-Associated Proteins	3,942

A.2 Performance Statistics

Table A.2: Performance statistics for SGPMAN8.0

Metric	Value
Total sequences read	151,066,931
Total valid PIMs	151,066,930
Validity rate	100.00%
Total bytes processed	72.17 GB
Processing time	13-14 hours
Average speed	3,141 seq/s
ESM2 model	esm2_t6_8M_UR50D
ESM2 embedding dim	320
Total feature dim	410

A.3 PIDP Tools

- metapredict: <https://github.com/idptools/metapredict>
- AIUPred: <https://github.com/idptools/aiupred>

A.4 APD Database

The Antimicrobial Peptide Database (APD) contains 3,306 antiviral peptides used for training the activity prediction model.

A.5 Version History

Table A.3: Version history of SGPMAN

Version	Key Features
v8.0	Transfer Learning (ESM2), Multi-Objective Predictor, Peptide Design Engine, Drug-Likeness Filters, Pareto Front Optimization, Active Learning, LoRA Fine-Tuning, Grassmann Multilevel
v17	Corrected wedge similarity, KL decomposition, PIDP integration, TherapeuticProfiler
v16	Grassmann projection, Fubini-Study metric, Ricci curvature
v15	Fractal dimension, Wasserstein distance, polarity Laplacian
v14	Radon transform, entropy-based metrics
v13	Specular reflection, Householder transformations
v12	Biological weighting, tri-vector calculations

COMPLETE SUMMARY OF ALL 18 MATHEMATICAL METRICS

This appendix provides a comprehensive summary of all 18 mathematical metrics implemented in SGPMAN8.0, including their symbols and descriptions.

Metric	Symbol	Description
Shannon Entropy	$H(v)$	Uncertainty in polarity distribution
Jensen-Shannon Divergence	$JS(v, w)$	Similarity between two distributions
Gini Coefficient	$Gini(v)$	Inequality of polarity distribution
Structural Complexity	$Complexity(v)$	Combined complexity index
Functional Modularity	$Modularity(v)$	Clustering of significant transitions
Moran's I	$Moran(v)$	Spatial autocorrelation
Hellinger Distance	$Hellinger(v, w)$	Distribution similarity
Spearman Correlation	$\rho(v, w)$	Rank correlation
Grassmann Projection	$d_{Gr}(v, w)$	Subspace distance
Fubini-Study	$d_{FS}(v, w)$	Projective space distance
Ricci Curvature	$\kappa(v, w)$	Geometric curvature
Wasserstein Distance	$W_1(v, w)$	Optimal transport distance
Fractal Dimension	$D_f(v)$	Self-similarity dimension
Radon Transform	$\mathcal{R}(v)$	Projection transform
Karhunen-Loève	λ_k	Principal components
Polarity Laplacian	$L(v)$	Connectivity structure
Commutator Norm	$[v, w]$	Non-commutativity measure
Anticommutator Similarity	$\{v, w\}$	Commutativity measure

COMPLETE CLASS AND FUNCTION REFERENCE

This appendix provides a comprehensive reference of all functions and classes available in SGPMAIN8.0.

C.1 Functions

Function	Description
compute_pim_profile(sequence, use_weights)	Converts sequence to PIM vector
compute_trimer_profile(sequence)	Computes trimer profile
wedge_product_oriented(v, w)	Computes oriented wedge product
wedge_product_with_ci(v, w, n_bootstrap, use_bootstrap)	Wedge product with confidence intervals
specular_reflection(v)	Applies specular reflection
is_specular_reflection_ga(v1, v2, threshold)	Checks reflection relationship
interior_product(v, subspace_name)	Projects onto subspace
interior_product_magnitude(v, subspace_name)	Magnitude of interior product
rotor_angle(v1, v2, plane_indices)	Computes rotor angle
find_optimal_plane(v1, v2)	Finds optimal rotation plane
find_rotation_angle(v1, v2, plane)	Finds rotation angle
general_rotor(v, target, n_steps)	Computes general rotor trajectory
commutator(v, w)	Computes commutator
commutator_norm(v, w)	Computes commutator norm
anticommutator(v, w)	Computes anticommutator
anticommutator_similarity(v, w)	Computes anticommutator similarity
dot_product_metric(v, w, metric)	Computes metric dot product
norm_metric(v, metric)	Computes metric norm
similarity_metric(v, w, metric)	Computes metric similarity
metric_signature_info()	Returns metric signature information
geometric_product_full(v, w, metric)	Computes full geometric product
geometric_product_decomposition(v, w, metric)	Computes functional-structural decomposition
hodge_dual(v, metric)	Computes Hodge dual
hodge_complementarity(v1, v2)	Computes Hodge complementarity
grassmann_distance(v1, v2)	Computes Grassmann distance
grassmann_geodesic(v1, v2, n_steps)	Computes Grassmann geodesic

Function	Description
clifford_signature(v)	Computes Clifford signature
clifford_distance(sig1, sig2)	Computes Clifford distance
grassmann_projection_distance(v1, v2)	Computes projection distance
grassmann_fubini_study(v1, v2)	Computes Fubini-Study metric
grassmann_ricci_curvature(v1, v2)	Computes Ricci curvature
karhunen_loeve_decomposition(vectors, n_components)	Performs KL decomposition
discrete_radon_transform(v, n_angles)	Computes Radon transform
fractal_dimension(v, scales)	Computes fractal dimension
wasserstein_distance(v1, v2)	Computes Wasserstein distance
polarity_interaction_laplacian(v)	Computes polarity Laplacian
shannon_entropy(v)	Computes Shannon entropy
jensen_shannon_divergence(v1, v2)	Computes Jensen-Shannon divergence
gini_coefficient(v)	Computes Gini coefficient
structural_complexity(v)	Computes structural complexity
functional_modularity(v, threshold)	Computes functional modularity
hellinger_distance(v1, v2)	Computes Hellinger distance
spearman_correlation(v1, v2)	Computes Spearman correlation
morans_i(v, weight_matrix)	Computes Moran's I
compute_enhanced_metrics(v1, v2)	Computes all enhanced metrics
grassmann_multilevel_distance(v1, v2, k)	Computes multilevel distance
grassmann_multilevel_similarity(v1, v2, k)	Computes multilevel similarity
grassmann_projection_asymmetry(v1, v2, k)	Computes projection asymmetry
grassmann_sectional_curvature(v1, v2, v3, k)	Computes sectional curvature
grassmann_sectional_curvature_sampled(vectors, k, n_samples)	Computes sampled curvature
grassmann_volume(vectors, k)	Computes Grassmann volume
grassmann_cycles(vectors, k, threshold)	Detects topological cycles
grassmann_karcher_mean(vectors, k)	Computes Karcher mean
grassmann_svd_similarity(v1, v2, k)	Computes SVD similarity
read_fasta_file(filepath)	Reads FASTA file
read_fasta_stream(filepath, verbose)	Streams FASTA file
pim_to_hash(pim_vector, tolerance)	Converts PIM to hash

C.2 Classes

Class	Description
DiskCache	Persistent disk caching for PIM vectors and ESM2 embeddings
SVDCache	Caching for SVD decompositions
ProcessingTracker	Tracks file processing progress
OnlineStatistics	Incremental statistics computation
ProgressiveSampler	Reservoir sampling for representative subsets
PIMHashIndex	LSH hash index for fast similarity search
GroupStatistics	Comprehensive group statistics
GrassmannPIM	Mathematical engine for geometric algebra operations
AdvancedGroupAnalyzer	Core analysis pipeline
ChemicalProfiler	Derives chemical properties from PIM vectors
PIDPProfiler	Intrinsic disorder prediction (metapredict + AIUPred)
TherapeuticProfiler	Synthetic peptide design
ChEMBLMapper	UniProt to ChEMBL mapping
APDLoader	Antimicrobial peptide database loader
EnhancedFeatureExtractor	Feature extraction with ESM2
MultiObjectivePredictor	Multi-objective peptide prediction
DrugLikenessFilter	Drug-likeness evaluation
PeptideGenerator	Bayesian optimization for peptide design
PeptideDesignEngine	Complete peptide design engine

BIBLIOGRAPHY

- [1] Polanco, C., Uversky, V.N., Huberman, A., et al. (2023). Bioinformatics-based Analysis of the Variability of MPOX Virus Proteins. *Current Bioinformatics*, 18(15), 170-191.
- [2] Hestenes, D., & Sobczyk, G. (1984). *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel Publishing Company.
- [3] Hestenes, D. (2017). The geometric algebra of Grassmann and Clifford. *Advances in Applied Clifford Algebras*, 27(1), 101-117.
- [4] Dorst, L., Fontijne, D., & Mann, S. (2007). *Geometric Algebra for Computer Science: An Object-Oriented Approach to Geometry*. Morgan Kaufmann.
- [5] Edelman, A., Arias, T. A., & Smith, S. T. (1998). The geometry of algorithms with orthogonality constraints. *SIAM Journal on Matrix Analysis and Applications*, 20(2), 303-353.
- [6] Fubini, G. (1904). Sulle metriche definite da una forma Hermitiana. *Atti del Reale Istituto Veneto di Scienze, Lettere ed Arti*, 63(2), 501-513.
- [7] Study, E. (1905). Kürzeste Wege im komplexen Gebiet. *Mathematische Annalen*, 60(3), 321-378.
- [8] Radon, J. (1917). Über die Bestimmung von Funktionen durch ihre Integralwerte längs gewisser Mannigfaltigkeiten. *Berichte über die Verhandlungen der Königlich-Sächsischen Gesellschaft der Wissenschaften zu Leipzig, Mathematisch-Physische Klasse*, 69, 262-277.
- [9] Mandelbrot, B. B. (1983). *The Fractal Geometry of Nature*. W. H. Freeman.
- [10] Householder, A. S. (1958). Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4), 339-342.
- [11] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423.
- [12] Jensen, J. L. W. V. (1906). Sur les fonctions convexes et les inégalités entre les valeurs moyennes. *Acta Mathematica*, 30(1), 175-193.
- [13] Gini, C. (1912). *Variabilità e mutabilità: contributo allo studio delle distribuzioni e delle relazioni statistiche*. Bologna: C. Cuppini.
- [14] Moran, P. A. P. (1950). Notes on continuous stochastic phenomena. *Biometrika*, 37(1/2), 17-23.

- [15] Hellinger, E. (1909). Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. *Journal für die reine und angewandte Mathematik*, 136, 210-271.
- [16] Spearman, C. (1904). The proof and measurement of association between two things. *American Journal of Psychology*, 15(1), 72-101.
- [17] Wasserstein, L. N. (1969). Markov processes over denumerable products of spaces describing large systems of automata. *Problems of Information Transmission*, 5(3), 47-52.
- [18] Karhunen, K. (1946). Über lineare Methoden in der Wahrscheinlichkeitsrechnung. *Annales Academiæ Scientiarum Fennicæ, Series A1: Mathematica-Physica*, 37, 1-79.
- [19] Loève, M. (1946). Fonctions aléatoires de second ordre. *Comptes Rendus de l'Académie des Sciences*, 222, 469-470.
- [20] Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (1997). Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1-3), 3-25.
- [21] Veber, D. F., Johnson, S. R., Cheng, H. Y., Smith, B. R., Ward, K. W., & Kopple, K. D. (2002). Molecular properties that influence the oral bioavailability of drug candidates. *Journal of Medicinal Chemistry*, 45(12), 2615-2623.
- [22] Rawat, P., et al. (2023). metapredict: A fast, accurate, and easy-to-use predictor of protein disorder. *Bioinformatics*, 39(2), btad098.
- [23] Erdos, G., et al. (2024). AIUPred: A biophysical model for protein disorder prediction. *Protein Science*, 33(2), e4832.
- [24] Lin, Z., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130.
- [25] Hu, E. J., et al. (2021). LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*.
- [26] Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.